Production RAG

Section 13 | Production RAG — Graphs, Multimodal Data & Data Hygiene

Author: Suresh D R | AI Product Developer & Technology Mentor

---

Table of Contents

Introduction — Why Vector-Only RAG Is Not Enough

Section 1 — Graph RAG

- 1.1 The Problem With a Normal Vector Database
- 1.2 What a Graph Database Is — Nodes, Edges and Properties
- 1.3 End-to-End: From Document to Graph to Answer
- 1.4 What Kind of Data Belongs in a Graph
- 1.5 Real-World Use Cases
- 1.6 Building the Graph — Entity and Relationship Extraction
- 1.7 Entity Resolution
- 1.8 Graph Retrieval Patterns
- 1.9 Microsoft GraphRAG — Community Summaries
- 1.10 Hybrid Graph + Vector RAG
- 1.11 Choosing a Graph Database
- 1.12 When NOT to Use a Graph

Section 2 — Where Does the Data Live? Vector DB vs Graph DB vs SQL

- 2.1 The Three Stores at a Glance
- 2.2 What Gets Stored in the Vector Database
- 2.3 What Gets Stored in the Graph Database

- 2.4 What Gets Stored in the SQL Database
- 2.5 One Document, Three Stores — Worked Example
- 2.6 How the Stores Are Linked
- 2.7 Which Store Answers Which Question

Section 3 — Multimodal Ingestion: Tables, Images and Everything Else

- 3.1 Document Parsing — The Step Everyone Skips
- 3.2 Tables
- 3.3 Images, Charts and Diagrams
- 3.4 Scanned Documents and OCR
- 3.5 Spreadsheets and CSV Files
- 3.6 Audio and Video
- 3.7 Code and Technical Documentation
- 3.8 The Unified Ingestion Pipeline
- 3.9 Step by Step: Extracting and Storing Each Data Type

Section 4 — Deduplication and Document Versioning

- The Core Scenario — The Same PDF Is Uploaded Again
- 4.1 Exact Duplicates — File Hashing
- 4.2 Same Content, Different File — Content Hashing
- 4.3 Near-Duplicates — MinHash and SimHash
- 4.4 Semantic Duplicates — Embedding Similarity
- 4.5 Chunk-Level Deduplication
- 4.6 Versioning — When the Same Document Is Updated
- 4.7 Record Manager — Tracking What Is Indexed
- 4.8 Retrieval-Time Deduplication — MMR

Section 5 — Metadata, Access Control and Multi-Tenancy

- 5.1 Metadata Schema Design
- 5.2 Document-Level Access Control

- 5.3 Multi-Tenancy

Section 6 — Freshness and Incremental Indexing

- 6.1 Incremental Indexing
- 6.2 Change Data Capture
- 6.3 Time-Aware Retrieval

Section 7 — When to Use What

INTRODUCTION — WHY VECTOR-ONLY RAG IS NOT ENOUGH

What Section 12 Solved

Section 12 made retrieval smarter — better chunks, better queries, better ranking, better answers. But every technique there assumed three things quietly:

1. The data is **text**.
2. The answer lives **inside a chunk**.
3. The data is **clean** — one copy of each document, always current, visible to everyone.

In real organisations, none of these assumptions hold. This section is about what breaks when they fail.

Failure 1: The Relationship Problem

What happens: The user's question is about how things are *connected*, not what a single document says. The connection is spread across many chunks, and no single chunk contains it.

Real example: "Which projects under the CTO use Python?" needs 4 facts from 4 different chunks. Vector search finds some of them, never the full chain. Answer: "I could not find this."

Why it happens: Embeddings measure *similarity*, not *connection*. (Explained in detail in Section 1.1.)

The fix: Graph RAG — store entities and relationships explicitly and traverse them.

Failure 2: The "Whole Dataset" Question

What happens: The user asks a question whose answer is a pattern across the entire corpus. No top-K retrieval can see the whole corpus.

```
User asks: "What are the main recurring themes in employee exit interviews?"

There are 312 exit interview records.
Vector search returns the 4 most similar chunks.
The answer is based on 4 of 312 interviews — 1.3% of the data.
```

The fix: Microsoft GraphRAG community summaries, or RAPTOR (Section 12, 1.7).

Failure 3: The Flattened Table Problem

What happens: A PDF contains a table. The text extractor reads it left-to-right, row by row, and the structure disappears.

```
Original table in HR policy PDF:

| Leave Type | Days     | Carry Forward |
|------------|----------|---------------|
| Earned     | 18       | Yes, up to 45 |
| Sick       | 10       | No            |
| Casual     | 6        | No            |

What a naive PDF extractor produces:

"Leave Type Days Carry Forward Earned 18 Yes, up to 45
 Sick 10 No Casual 6 No"

User asks: "Can sick leave be carried forward?"
The LLM sees "Sick 10 No Casual 6 No" — which "No" belongs to what?
It might guess correctly. It might not.
```

The fix: Layout-aware parsing and table-specific indexing (Section 3.2).

Failure 4: The Invisible Image Problem

What happens: Critical information is inside an image — an org chart, a flowchart of the leave approval process, a revenue chart. The text extractor skips images completely. The information never enters the vector database.

```
TechNova's Annual Report, page 7: a bar chart of revenue by quarter.
Text extractor output for page 7: "Figure 3: Quarterly Revenue"

User asks: "Which quarter had the highest revenue?"
The only indexed text is the caption. The answer is lost.
```

The fix: Image captioning with vision models, multimodal embeddings, or page-image retrieval (Section 3.3).

Failure 5: The Duplicate Document Problem

What happens: The same HR policy is uploaded three times — by HR, by a manager, and by an employee who downloaded and re-uploaded it. The vector database now has three identical copies of every chunk.

User asks: "What is the sick leave policy?"

Rank	Source File	Chunk Text	Useful?
1	HR_Policy_2024.pdf	"Sick leave is 10 days per year..."	Yes
2	HR_Policy_2024 (1).pdf	"Sick leave is 10 days per year..."	Duplicate
3	HR Policy - final.pdf	"Sick leave is 10 days per year..."	Duplicate
4	HR_Policy_2024.pdf	"Medical certificate required..."	Yes

Three of four retrieval slots are wasted on the same text. The chunk about how to apply for sick leave never makes it into the top 4.

The fix: Hashing, near-duplicate detection and a record manager (Section 4).

Failure 6: The Stale Version Problem

What happens: The policy is updated. The new version is indexed. The old version is never deleted. Now the database contains two contradicting answers.

```
HR_Policy_2024.pdf: "Sick leave is 10 days per year."
HR_Policy_2025.pdf: "Sick leave is 12 days per year."   (April 2025 revision)

User asks: "How many sick days do I get?"
Retriever returns both chunks. The LLM either picks one at random,
or says "10 or 12 days" — both are wrong answers for the user.
```

The fix: Versioning with upserts and stale-chunk cleanup (Section 4.6).

Failure 7: The Wrong Person Sees the Wrong Data

What happens: The salary band document and the policy handbook are in the same vector database. An L2 engineer asks a question and receives a chunk containing the CEO's compensation.

The fix: Metadata-based access control (Section 5).

The Summary

Vector-Only RAG Failure	Root Cause	Fix
Multi-hop relationship questions fail	Similarity ≠ connection	Graph RAG
"Whole dataset" questions fail	Top-K sees a tiny slice	GraphRAG community summaries

Vector-Only RAG Failure	Root Cause	Fix
Tables become word soup	Structure lost in extraction	Layout-aware parsing, table indexing
Charts and diagrams ignored	Text extractor skips images	Vision captioning, multimodal embeddings
Same chunk retrieved 3 times	Duplicate uploads	Hashing, record manager, MMR
Contradicting answers	Old versions never deleted	Versioning and upserts
Confidential data leaked	No access filtering	Metadata access control
Answers based on last quarter's numbers	No freshness strategy	Incremental indexing, CDC

> Section 12 made RAG **smart**. This section makes RAG **production-ready** — correct on real data, safe for real users, and measurable.

SECTION 1 — GRAPH RAG

Why This Section Is Required

Think of how you actually learn how a company works. You do not read every document. You learn *who knows whom*, *who owns what*, *what depends on what*. Organisations are networks. A lot of the most valuable questions — "who should I ask about X?", "what breaks if this service goes down?", "which suppliers affect this product?" — are questions about the network, not about any one document.

Vector databases store documents as isolated points in space. Graph databases store the network itself. Graph RAG gives your LLM access to that network.

Simple analogy: A vector database is like a pile of business cards — each card tells you about one person. A graph database is like the company's LinkedIn — it shows who works with whom, who reports to whom, and who worked on what together. To answer "who is my manager's manager's peer?", the pile of cards is useless. The network answers it instantly.

What it is: Using a graph database (nodes and edges) as a retrieval source for RAG, either on its own or combined with a vector database.

1.1 The Problem With a Normal Vector Database

Simple Analogy

Imagine cutting a company's org chart into 100 small paper slips, one sentence per slip, and throwing them into a box. Now someone asks, "Who is Karan's manager's manager?" You can pull out slips that *mention* managers, but the box has no idea which slip connects to which. The connections were in the picture, and cutting it into slips destroyed them. A vector database is that box of slips.

First, How a Vector Database Sees Your Data

Step	What Happens	What Is Lost
1. Chunking	The document is cut into small pieces of text	Links between sentences in different pieces
2. Embedding	Each piece becomes a list of numbers that captures its *meaning*	Exact facts, names and direction
3. Storing	Each vector is saved as an independent point	Any knowledge that piece 14 and piece 37 are about the same person
4. Searching	The question's vector is compared with the stored vectors; the top K most *similar* pieces come back	Pieces that are *connected* to the answer but not *similar* to the question

So a vector database knows **what each chunk means**. It does **not** know **how the things inside those chunks are connected**.

What a Vector Database Knows	What a Vector Database Does NOT Know
The text of every chunk	That "Karan" in Chunk 14 and "Karan" in Chunk 37 are the same person
Which chunks are similar in meaning	Which chunks are linked by facts
Roughly what topic a chunk is about	Direction: whether Karan reports to Priya or Priya reports to Karan
The top K closest chunks to a question	All the chunks needed to follow a chain of facts

---

The Six Problems, One by One

Problem 1: Similarity Is Not Connection

What happens: The facts needed to answer a question are connected, but they are not similar to each other or to the question.

Chunk	Text	Similar to the Question "Which projects under the CTO use Python?"
14	"Karan Bhatia reports to Priya, the CTO."	Somewhat (mentions CTO)
37	"The Platform team is led by Karan Bhatia."	No
52	"Project Orion is owned by the Platform team."	Somewhat (mentions project)
88	"Orion's backend is written in Python and Go."	Somewhat (mentions Python)

Chunk 37 is the critical link in the chain, and it looks completely unrelated to the question. Vector search will almost never retrieve it.

Problem 2: Every Chunk Is an Island

What happens: A fact that is split across chunks, or that uses a pronoun, loses its meaning.

Chunk	Text	Problem
21	"Karan Bhatia joined as Engineering Manager in 2022."	Fine
22	"**He** reports directly to the CTO and manages 14 engineers."	Who is "he"? Chunk 22 alone does not say

The question "How many engineers does Karan manage?" matches Chunk 22 only weakly, because Chunk 22 never says "Karan".

Problem 3: The Top-K Limit

What happens: A question that needs 4 facts from 4 different chunks must get **all 4** into the top K. Each extra hop makes that much less likely.

Hops Needed	Chunks That Must All Be Retrieved	Chance of Success With Vector Search
1	1	High

Hops Needed	Chunks That Must All Be Retrieved	Chance of Success With Vector Search
2	2	Medium
3	3	Low
4+	4+	Very low

A **hop** is one step along a relationship.

Example — a 4-hop question: "Which technologies are used by projects under the CTO?"

Hop	From	Relationship Followed	Result
1	CTO (Priya)	Who `REPORTS_TO` the CTO?	Karan
2	Karan	Which team does he `LEAD`?	Platform
3	Platform	Which projects does it `OWN`?	Orion
4	Orion	Which technologies does it `USE`?	Python, Go

Approach	What It Needs	Outcome
Vector RAG	All 4 separate chunks in the top-K at once	Probability drops sharply with every extra hop — usually fails
Graph RAG	One Cypher query that follows all 4 edges	Exact answer in milliseconds

```
MATCH (cto:Person {role: 'CTO'})<-[:REPORTS_TO]-(m:Person)
      -[:LEADS]->(t:Team)-[:OWNS]->(p:Project)-[:USES]->(tech:Technology)
RETURN p.name AS project, collect(tech.name) AS technologies
```

```
Result:
  project: "Orion",  technologies: ["Python", "Go"]
```

Problem 4: Direction and Relationship Type Are Blurred

What happens: Embeddings capture *topic*, not *who did what to whom*.

Sentence A	Sentence B	Meaning	Embedding Similarity
"Karan reports to Priya."	"Priya reports to Karan."	Opposite	Very high — almost the same words
"Karan manages Rahul."	"Rahul manages Karan."	Opposite	Very high

A vector database cannot reliably tell these apart. A graph stores the arrow's direction explicitly, so it never confuses them.

Problem 5: "List All" and "Count" Questions Over Relationships

What happens: "Who are **all** the people under Priya?" — 45 people, mentioned across 30 different chunks. Vector search returns 4 or 5 chunks, so the answer lists 6 people and silently misses 39.

Problem 6: No Explanation of *Why*

What happens: Vector search returns chunks with similarity scores like 0.82. It cannot say *why* an answer is correct. A graph returns a **path**: "Orion ← owned by Platform ← led by Karan → reports to Priya". The path *is* the explanation.

Summary: Vector Database vs Graph Database on the Same Questions

Question	Vector Database Result	Graph Database Result
"What does the leave policy say about sick leave?"	Excellent	Not needed
"Who does Karan report to?"	Works if one chunk says it clearly	Instant, exact
"Who is two levels above Karan?"	Often fails (needs 2 chunks)	Instant, exact
"Which projects under the CTO use Python?"	Usually fails (needs 4 chunks)	Instant, exact
"List everyone under Priya"	Partial list	Complete list

Question	Vector Database Result	Graph Database Result
"Does Priya report to Karan?"	May wrongly say yes	Correctly says no
"How is Orion connected to the CEO?"	Almost always fails	Returns the full path

The Core Idea of the Fix

> Instead of hoping the right chunks land together in the top K, **pull the facts out of the text once, at indexing time, and store the connections explicitly.** Then, at question time, *follow* the connections instead of *guessing* them. That is exactly what a graph database does.

1.2 What a Graph Database Is — Nodes, Edges and Properties

Simple Analogy

The Namma Metro map in Bengaluru. Each **station** is a circle (a node). Each **line between two stations** is a connection (an edge). Each station has **details** — name, interchange or not, opening year (properties). To plan a trip, you don't read a description of every station; you follow the lines. A graph database is a metro map for your company's knowledge.

A graph database has only three building blocks. Once you understand these three, you understand graph databases.

Building Block 1: Node — "The Thing"

A **node** is one real-world thing: a person, a team, a project, a technology, a policy, a document. Think of a node as a **noun**.

Every node has a **label** that says what *kind* of thing it is.

Label (Kind of Thing)	Example Nodes
`Person`	Arjun, Priya, Karan, Rahul
`Team`	Platform, Data & AI, HR
`Project`	Orion, Atlas

Label (Kind of Thing)	Example Nodes
`Technology`	Python, Go, Kubernetes
`Policy`	Leave Policy, Travel Policy
`Location`	Bengaluru, Pune

Key rule: One real-world thing = **one** node, no matter how many documents mention it. Priya might appear in 200 documents, but she is **one** node in the graph. This is what fixes Problem 2 (islands).

Building Block 2: Edge (Relationship) — "The Connection"

An **edge** connects two nodes and says *how* they are related. Think of an edge as a **verb**.

Every edge has three things:

Part of an Edge	Meaning	Example
Start node	Who or what the connection comes from	Karan
Type	What kind of connection	`REPORTS_TO`
End node	Who or what it goes to	Priya
Direction	Always from start → end	Karan → Priya (not Priya → Karan)

Written in graph notation: `(Karan)-[:REPORTS_TO]->(Priya)`

Direction matters — this fixes Problem 4:

Edge	Meaning
`(Karan)-[:REPORTS_TO]->(Priya)`	Karan's manager is Priya
`(Priya)-[:REPORTS_TO]->(Karan)`	Priya's manager is Karan — a completely different fact

Common edge types at TechNova:

Edge Type	From → To	Reads As
`REPORTS_TO`	Person → Person	"Karan reports to Priya"
`LEADS`	Person → Team	"Karan leads Platform"
`MEMBER_OF`	Person → Team	"Rahul is a member of Platform"
`OWNS`	Team → Project	"Platform owns Orion"
`USES`	Project → Technology	"Orion uses Python"
`LOCATED_IN`	Person → Location	"Karan is located in Bengaluru"
`MENTIONED_IN`	Any node → Chunk	"Karan is mentioned in Chunk 14"

Building Block 3: Property — "The Details"

A **property** is a key–value detail stored on a node *or* on an edge.

Stored On	Example	Properties
Node	Priya	`{name: "Priya Sharma", role: "CTO", employee_id: "EMP0007", joined: 2019}`
Node	Orion	`{status: "Behind schedule", start_date: "2024-06-01"}`
Edge	Karan → REPORTS_TO → Priya	`{since: 2022}`
Edge	Rahul → MEMBER_OF → Platform	`{role: "Senior Engineer", allocation: "100%"}`

Simple rule for deciding node vs property:

If the Detail…	Make It a…	Example
Is a simple value that only describes one thing	Property	Priya's joining year
Is itself a thing that other things connect to	Node	Bengaluru (many people are located there)

Putting It Together: Every Edge Is One Sentence

A graph is simply a collection of small facts, each written as **subject → verb → object**. These are called **triples**.

Subject (Node)	Verb (Edge)	Object (Node)	As a Sentence
Karan	`REPORTS_TO`	Priya	Karan reports to Priya
Priya	`REPORTS_TO`	Arjun	Priya reports to Arjun
Karan	`LEADS`	Platform	Karan leads the Platform team
Platform	`OWNS`	Orion	Platform owns Project Orion
Orion	`USES`	Python	Orion uses Python
Orion	`USES`	Go	Orion uses Go

Drawn as a picture:

```
(Arjun:Person {role: "CEO"})
        ▲
        | REPORTS_TO
        |
(Priya:Person {role: "CTO"})
        ▲
        | REPORTS_TO
        |
(Karan:Person {role: "Engineering Manager"})
        |
        | LEADS
        ▼
(Platform:Team)
        |
        | OWNS
        ▼
(Orion:Project {status: "Behind schedule"})
        |
```

```
          | USES
          ├──────────────┐
          ▼              ▼
(Python:Technology)  (Go:Technology)
```

How a Graph Database Searches: Traversal

This is the most important difference from a vector database.

	Vector Database	**Graph Database**
How it searches	Compares the question with stored vectors and returns the most *similar*	Starts at one node and *walks* along edges to connected nodes
Analogy	Asking a librarian for "books like this one"	Following metro lines from station to station
What decides the result	Similarity score	Actual connections
Cost of one more hop	Needs one more chunk to land in the top K — getting harder each time	One more step along an edge — takes milliseconds
Result	Chunks + similarity scores	Nodes, edges and paths

Graph databases are built so that every node directly points to its neighbours. Following an edge is like the metro train moving to the next station — it does not need to search the whole city to find the next stop. That is why multi-hop questions are fast.

Cypher in 60 Seconds

Cypher is the query language of Neo4j and several other graph databases. It is designed to *look* like the picture: nodes in round brackets `( )`, edges in square brackets `[ ]`, arrows for direction.

What You Want	Cypher	Result
Create a node	`CREATE (:Person {name: 'Karan', role: 'Engineering Manager'})`	Karan node exists
Create an edge	`MATCH (k:Person {name:'Karan'}), (p:Person {name:'Priya'}) CREATE (k)-[:REPORTS_TO {since: 2022}]->(p)`	Karan → Priya
One hop	`MATCH (:Person {name:'Karan'})-[:REPORTS_TO]->(m) RETURN m.name`	Priya
Two hops	`MATCH (:Person {name:'Karan'})-[:REPORTS_TO]->()-[:REPORTS_TO]->(m) RETURN m.name`	Arjun
Everyone under Priya, any depth	`MATCH (e)-[:REPORTS_TO*]->(:Person {name:'Priya'}) RETURN e.name`	Complete list
Path between two nodes	`MATCH p = shortestPath((:Project {name:'Orion'})-[*]-(:Person {role:'CEO'})) RETURN p`	Orion … Arjun

The `*` in `[:REPORTS_TO*]` means "follow this edge type as many times as needed". This one symbol solves Problem 5 (list all), which vector search cannot solve at all.

1.3 End-to-End: From Document to Graph to Answer

Simple Analogy

Think of a new HR executive building an org chart on their first week. They **read** the documents, **note down** every name and who works with whom, **check** that "Priya" and "the CTO" are the same person, **draw** the chart on the whiteboard, and keep the **original files** in a drawer for reference. Later, when someone asks a question, they **look at the chart**, **trace the lines**, and **open the file** to confirm. Graph RAG does exactly this, automatically.

PART A — Indexing (Done Once, When Documents Arrive)

Step 1: Source documents

Document	Text
Org_Update_2024.pdf	"Karan Bhatia leads the Platform team. He reports directly to Priya Sharma, the CTO."

Document	Text
Leadership_Note.docx	"The CTO reports to Arjun, our CEO."
Orion_Status_Q3.pdf	"Project Orion is owned by Platform. Its backend is written in Python and Go. Orion is 6 weeks behind schedule due to the payments migration."

Step 2: Chunk the documents — exactly as in normal RAG. Each chunk gets an ID (C1, C2, C3).

Step 3: Extract entities and relationships with an LLM (details in 1.6)

Chunk	Entities Found (Nodes)	Relationships Found (Edges)
C1	Karan Bhatia (Person), Platform (Team), Priya Sharma (Person, CTO)	Karan `LEADS` Platform; Karan `REPORTS_TO` Priya Sharma
C2	The CTO (Person), Arjun (Person, CEO)	The CTO `REPORTS_TO` Arjun
C3	Orion (Project), Platform (Team), Python, Go (Technology)	Platform `OWNS` Orion; Orion `USES` Python; Orion `USES` Go

Notice that the LLM resolved "He" in C1 to Karan. That fixes Problem 2.

Step 4: Entity resolution — merge duplicates (details in 1.7)

Extracted Names	Same Real Thing?	Merged Into
"Priya Sharma", "The CTO"	Yes (role = CTO)	One node: Priya Sharma
"Platform" (C1), "Platform" (C3)	Yes	One node: Platform

Step 5: Write nodes and edges into the graph database

```
MERGE (k:Person {name: 'Karan Bhatia'})
MERGE (p:Person {name: 'Priya Sharma'}) SET p.role = 'CTO'
MERGE (a:Person {name: 'Arjun'})       SET a.role = 'CEO'
MERGE (t:Team {name: 'Platform'})
MERGE (o:Project {name: 'Orion'})      SET o.status = '6 weeks behind'
MERGE (py:Technology {name: 'Python'})
MERGE (go:Technology {name: 'Go'})
```

```
MERGE (k)-[:LEADS]->(t)
MERGE (k)-[:REPORTS_TO]->(p)
MERGE (p)-[:REPORTS_TO]->(a)
MERGE (t)-[:OWNS]->(o)
MERGE (o)-[:USES]->(py)
MERGE (o)-[:USES]->(go)
```

`MERGE` means "create it only if it does not already exist" — so running this twice never creates duplicate nodes.

Step 6: Store the chunks in the vector database and link them to the graph

Store	What Is Saved
Vector database	C1, C2, C3 — text + embedding + metadata
Graph database	`(Karan)-[:MENTIONED_IN]->(C1)`, `(Orion)-[:MENTIONED_IN]->(C3)`, …

These `MENTIONED_IN` links let the system jump from a fact in the graph back to the original sentence as evidence.

PART B — Answering (Done for Every Question)

Step 7: The user asks a question

> "Why is Project Orion delayed, and who is the most senior person accountable for it?"

Step 8: Find the starting point in the graph

Action	Result
LLM extracts entities from the question	"Project Orion"
Look up the node	`(Orion:Project)` found

Step 9: Traverse the graph

```
MATCH path = (o:Project {name: 'Orion'})<-[:OWNS]-(:Team)<-[:LEADS]-(:Person)
             -[:REPORTS_TO*]->(top:Person)
WHERE NOT (top)-[:REPORTS_TO]->()
RETURN path
```

Hop	From	Edge	To
1	Orion	owned by (`OWNS` , reversed)	Platform
2	Platform	led by (`LEADS` , reversed)	Karan
3	Karan	`REPORTS_TO`	Priya (CTO)
4	Priya	`REPORTS_TO`	Arjun (CEO)

Step 10: Fetch the supporting text — follow `MENTIONED_IN` from Orion to C3, and run a normal vector search for "Orion delay reason". Both return: *"Orion is 6 weeks behind schedule due to the payments migration."*

Step 11: Send both to the LLM

Context Type	Content
Graph path	Orion ← Platform ← Karan → Priya (CTO) → Arjun (CEO)
Text evidence	"Orion is 6 weeks behind schedule due to the payments migration." (Orion_Status_Q3.pdf)

Step 12: Final answer

> "Project Orion is 6 weeks behind schedule because of the payments migration (Orion_Status_Q3.pdf). Orion is owned by the Platform team, led by Karan Bhatia, who reports to Priya Sharma (CTO), who reports to Arjun (CEO). The most senior person accountable is Arjun; the direct technical escalation is Priya."

The Whole Flow in One Table

#	Stage	Done By	Output
1	Collect documents	Ingestion pipeline	Raw files
2	Chunk	Text splitter	Chunks with IDs
3	Extract entities and relationships	LLM	Candidate nodes and edges
4	Resolve duplicates	Rules + fuzzy matching + LLM	Clean, merged nodes
5	Load the graph	Graph database (`MERGE`)	Nodes, edges, properties
6	Store and link chunks	Vector DB + `MENTIONED_IN` edges	Text evidence linked to graph
7	Receive question	Application	User question
8	Find start node	LLM entity extraction + lookup	Anchor node
9	Traverse	Cypher query	Path / subgraph
10	Fetch evidence	`MENTIONED_IN` + vector search	Supporting chunks
11	Build context	Application	Path + text
12	Generate answer	LLM	Answer with reasoning path and sources

> **In one line:** A vector database finds text that *sounds like* the question. A graph database *follows the actual connections* to the answer. Graph RAG uses the graph to find **what is connected** and the vector database to explain **why**.

1.4 What Kind of Data Belongs in a Graph

Simple Analogy

Not every kitchen needs a pressure cooker. If you only boil water, a kettle is enough. A graph database is a specialised tool — powerful when your data is highly connected, overkill when it is not.

The Test: Use a Graph When…

Signal	Why It Points to a Graph
Relationships are the question	"Who reports to…", "What depends on…", "Who is connected to…"
Questions need 2+ hops	Chains of connected facts
Entities repeat across many documents	The same person, product or supplier appears in 50 documents — a graph merges them into one node
The schema keeps evolving	New relationship types can be added without redesigning tables
Paths matter	"What is the shortest route between A and B?"
You need to explain *why*	A path through the graph is a human-readable explanation

Data Types That Fit Graphs Naturally

Data Type	Nodes	Edges
Org charts and HR hierarchy	Employees, teams, departments	REPORTS_TO, MEMBER_OF, MANAGES
IT infrastructure	Services, servers, databases	DEPENDS_ON, CALLS, HOSTED_ON
Supply chain	Suppliers, parts, products, factories	SUPPLIES, CONTAINS, SHIPS_TO
Financial transactions	Accounts, people, devices	TRANSFERRED_TO, OWNS, LOGGED_IN_FROM
Biomedical knowledge	Drugs, diseases, genes, proteins	TREATS, INTERACTS_WITH, ASSOCIATED_WITH
Legal documents	Cases, statutes, judges, clauses	CITES, OVERRULES, AMENDS
Regulations and compliance	Regulations, policies, controls, owners	REQUIRES, IMPLEMENTED_BY, OWNED_BY
Customer 360	Customers, orders, tickets, products	PURCHASED, RAISED, RELATED_TO
Research literature	Papers, authors, topics	CITES, WROTE, ABOUT

Data That Does NOT Fit Graphs

Data Type	Better Home
Long-form policy text, FAQs	Vector database
Numeric aggregations ("average salary by department")	SQL database

Data Type	Better Home
Time-series metrics	Time-series database
Documents with few shared entities	Vector database

1.5 Real-World Use Cases

Use Case 1: Enterprise Knowledge and "Who Knows What"

```
User asks: "Who at TechNova has worked on Kubernetes migrations
            and is currently in the Bengaluru office?"

Graph path:
(Person)-[:WORKED_ON]->(Project)-[:USES]->(Kubernetes)
(Person)-[:LOCATED_IN]->(Bengaluru)

Answer: a list of people with the projects that qualify them —
        and the path explains why each person was chosen.
```

Use Case 2: IT Incident Root-Cause Analysis

```
Alert: "Payments API is returning errors."

Graph question: "What does the Payments API depend on, and which of
                 those dependencies had a change deployed today?"

(PaymentsAPI)-[:DEPENDS_ON*1..3]->(dep)<-[:DEPLOYED_TO]-(change {date: today})

Answer: "Payments API → Auth Service → Redis Cluster 2.
         Redis Cluster 2 received a config change at 10:42 today."
```

The `*1..3` means "follow DEPENDS_ON edges between 1 and 3 hops deep" — something vector search cannot express at all.

Use Case 3: Fraud Detection

Fraud rarely shows up in a single transaction. It shows up as a *pattern of connections* — five "unrelated" accounts sharing one phone number and one device.

```
(Account A)-[:USES_DEVICE]->(Device X)<-[:USES_DEVICE]-(Account B)
(Account B)-[:HAS_PHONE]->(Phone 98xxxx)<-[:HAS_PHONE]-(Account C)

An LLM investigator asks: "Is this account connected to any
                           previously flagged accounts?"
Graph traversal finds the ring. Vector search never would.
```

Use Case 4: Healthcare and Drug Interactions

```
User asks: "Patient is on Warfarin. Is it safe to prescribe Drug Y?"

(Warfarin)-[:INTERACTS_WITH {severity: "high"}]->(Drug Y)

The graph returns a hard, explicit fact — not an embedding guess.
In medicine, an explicit relationship is far safer than a
"semantically similar" chunk.
```

Use Case 5: Compliance Mapping

```
User asks: "Which of our internal controls satisfy the data-retention
            clause of the DPDP Act, and who owns each control?"

(Regulation:DPDP)-[:HAS_CLAUSE]->(Clause:Retention)
      <-[:SATISFIES]-(Control)-[:OWNED_BY]->(Person)
```

Use Case 6: Legal Research

```
User asks: "Which judgments have cited Case X and later been overruled?"
```

```
(Case)-[:CITES]->(Case X)
(Case)<-[:OVERRULES]-(Later Case)
```

1.6 Building the Graph — Entity and Relationship Extraction

Simple Analogy

Reading a newspaper with two highlighters. The yellow highlighter marks every *name* — people, companies, places. The pink highlighter marks every *connection* — "acquired", "works for", "located in". At the end, you draw the yellow items as boxes and the pink items as arrows. That drawing is your knowledge graph. An LLM does this highlighting automatically.

What It Is

Most organisational knowledge is unstructured text. To build a graph from it, you use an LLM to extract **entities** (nodes) and **relationships** (edges) from each chunk.

How It Works

```
Step 1: Chunk the documents (as in normal RAG).

Step 2: Define the schema — which entity types and relationship
        types you care about. This keeps extraction consistent.

        Entities:      Person, Team, Project, Technology, Policy
        Relationships: REPORTS_TO, LEADS, OWNS, USES, GOVERNED_BY

Step 3: For every chunk, send it to an LLM with an extraction prompt:

        "Extract entities and relationships from the text below.
         Only use these entity types: [...]
         Only use these relationship types: [...]
         Return JSON."

Step 4: Input chunk:
```

```
        "Karan Bhatia leads the Platform team, which owns Project Orion.
         Karan reports directly to Priya, the CTO."

        LLM output:
        {
          "entities": [
            {"name": "Karan Bhatia", "type": "Person"},
            {"name": "Platform",      "type": "Team"},
            {"name": "Orion",         "type": "Project"},
            {"name": "Priya",         "type": "Person", "role": "CTO"}
          ],
          "relationships": [
            {"from": "Karan Bhatia", "type": "LEADS",      "to": "Platform"},
            {"from": "Platform",     "type": "OWNS",       "to": "Orion"},
            {"from": "Karan Bhatia", "type": "REPORTS_TO", "to": "Priya"}
          ]
        }

Step 5: Write nodes and edges into the graph database using MERGE
        (creates the node only if it does not already exist).

Step 6: Link each node back to its source chunk:
        (Karan)-[:MENTIONED_IN]->(Chunk 14)
        This lets you return the original text as evidence.
```

Schema-First vs Schema-Free Extraction

Approach	How	Pros	Cons
Schema-first	You define allowed entity and relationship types	Clean, consistent, queryable graph	Misses things outside the schema
Schema-free	LLM invents types as it goes	Captures everything	Messy — "works_for", "employed_by", "is_employee_of" all appear as separate types

Recommendation: Start schema-first for production. Use schema-free only for exploration, then turn what you learn into a schema.

Trade-off

Graph extraction needs one LLM call per chunk. For a 25-page document with 80 chunks, that is 80 extraction calls — more expensive than embedding. For a 10,000-document corpus, extraction cost and time become a real project budget item.

1.7 Entity Resolution

Simple Analogy

At a wedding, three guests talk about "Priya", "Priya S" and "the CTO". A good host realises they are all talking about the same person. Without that realisation, the guest list would show three different people. Entity resolution is the host who merges them.

What It Is

Deciding when two extracted entities refer to the same real-world thing, and merging them into one node.

Why It Is Required

Without entity resolution, the graph fragments. Relationships attached to "Priya" and relationships attached to "Priya Sharma" live on different nodes. Traversals stop halfway.

```
Before entity resolution:

(Karan)-[:REPORTS_TO]->(Priya)
(Priya Sharma)-[:REPORTS_TO]->(Arjun)
(The CTO)-[:APPROVED]->(Cloud Budget)

Question: "Who approved the cloud budget, and who do they report to?"
Traversal from "The CTO" finds no REPORTS_TO edge. Answer fails.

After entity resolution:

(Priya Sharma {role: "CTO", aliases: ["Priya", "The CTO"]})
    -[:REPORTS_TO]->(Arjun)
    -[:APPROVED]->(Cloud Budget)
```

```
(Karan)-[:REPORTS_TO]->(Priya Sharma)

Traversal works end to end.
```

How It Works

Method	How	Good For
Exact normalisation	Lowercase, strip titles and punctuation	"Dr. Priya Sharma" = "priya sharma"
Alias dictionaries	Maintain a known list — roles, abbreviations	"CTO" → Priya Sharma, "HR" → Human Resources
Fuzzy string matching	Edit distance between names	"Karan Bhatia" vs "Karan Bhatiya"
Embedding similarity	Embed entity name + context, merge above a threshold	"Platform team" vs "Platform Engineering"
LLM judgement	Ask an LLM "Are these the same entity?" for borderline pairs	Ambiguous cases
Unique IDs	Match on employee ID, product SKU, GSTIN	Structured sources — the most reliable of all

Rule of thumb: Whenever a reliable ID exists (employee ID, SKU), resolve on the ID. Use fuzzy and LLM methods only when there is no ID.

1.8 Graph Retrieval Patterns

Simple Analogy

There are different ways to use a city map. Sometimes you look up one address (lookup). Sometimes you explore the neighbourhood around a place (local expansion). Sometimes you plan a route between two places (path finding). Graph retrieval has the same modes.

Pattern 1: Text-to-Cypher (covered in Section 12, 2.3)

The LLM writes a Cypher query from the question. Best for precise, structured questions when the schema is well known.

Key production tip: Always give the LLM the graph schema in the prompt, and validate the generated query before running it. Run queries with a read-only database user so a badly generated query can never modify or delete data.

Pattern 2: Entity-Anchored Neighbourhood Expansion (Local Search)

```
Step 1: Extract entities from the question.
        "What is Karan working on?" → entity: Karan

Step 2: Find that entity's node in the graph.

Step 3: Pull its neighbourhood — all nodes within 1–2 hops.
        Karan → LEADS → Platform → OWNS → Orion
        Karan → REPORTS_TO → Priya
        Karan → MENTIONED_IN → Chunk 14, Chunk 37

Step 4: Convert the subgraph to text and send it to the LLM with
        the linked source chunks:
        "Karan leads Platform. Platform owns Orion.
         Karan reports to Priya (CTO)."
```

No Cypher generation needed — more robust than Text-to-Cypher for open-ended questions.

Pattern 3: Path Finding

```
User asks: "How is Project Orion connected to the CEO?"

MATCH p = shortestPath(
  (o:Project {name: 'Orion'})-[*..6]-(ceo:Person {role: 'CEO'})
)
RETURN p

Result: Orion ← OWNS ← Platform ← LEADS ← Karan
        → REPORTS_TO → Priya → REPORTS_TO → Arjun
```

Pattern 4: Global Search over Community Summaries

For "whole dataset" questions. Covered next in 1.9.

1.9 Microsoft GraphRAG — Community Summaries

Simple Analogy

Imagine a large city. You could describe it street by street — but no one can answer "what is this city like?" that way. Instead, the city is divided into neighbourhoods, and each neighbourhood has a short description: "Koramangala — startups and cafés", "Jayanagar — residential, old Bengaluru". Now "what is this city like?" is easy — read the neighbourhood descriptions. GraphRAG builds neighbourhoods (communities) inside your knowledge graph and writes a description for each.

What It Is

GraphRAG is Microsoft's open-source approach that:

1. Builds a knowledge graph from documents using LLM extraction.
2. Detects **communities** — clusters of tightly connected entities — using a graph clustering algorithm (Leiden).
3. Writes an LLM **summary for every community**, at multiple levels (small communities inside bigger ones).
4. Answers questions using those summaries.

The Problem It Solves

"What are the main themes in 312 exit interviews?" — standard RAG sees 4 chunks. GraphRAG sees the summaries of every theme-community, which together cover the entire dataset.

How It Works

```
INDEXING
Step 1: Extract entities + relationships from all 312 interviews.
Step 2: Build the graph. Entities: reasons, teams, managers, benefits.
Step 3: Run community detection.

  Community 1: {long hours, weekend deployments, on-call, burnout}
  Community 2: {Data & AI team, market salary, competitor offers}
  Community 3: {manager feedback, career growth, promotion delay}
  Community 4: {relocation, hybrid work, Bengaluru commute}

Step 4: LLM writes a summary for each community:

  Community 2 summary:
```

```
   "Compensation-driven exits are concentrated in the Data & AI
    practice, where most leavers cite competitor offers 25-40% above
    current pay. This aligns with Data & AI's 14.7% attrition - the
    highest in the company."

QUERYING - Global Search (map-reduce)
Step 5 (Map):    Ask the question against EVERY community summary
                 separately. Each returns a partial answer + an
                 importance score.
Step 6 (Reduce): Combine the highest-scoring partial answers into one
                 final answer.

Answer: "Exits cluster into four themes: workload and burnout,
         compensation (concentrated in Data & AI), career growth
         and promotion delays, and commute/hybrid-work concerns."
```

Local Search vs Global Search

Mode	Best For	How
Local search	Questions about specific entities — "What is Karan working on?"	Starts from matching entities, expands neighbourhood, adds linked chunks
Global search	Questions about the whole corpus — "What are the main themes?"	Map-reduce over community summaries

Trade-off

GraphRAG indexing is **expensive** — many LLM calls for extraction plus summarisation for every community. Global search is also slower, since it asks the question against many summaries. Worth it when "big picture" questions over large corpora are a core use case. Lighter variants (for example LightRAG) reduce indexing cost by simplifying the graph and summarisation steps.

1.10 Hybrid Graph + Vector RAG

Simple Analogy

A detective uses two things: a case file (detailed written statements) and an evidence board (photos connected with red string). The case file has the *details*. The board shows the *connections*. Good detectives use both. Hybrid RAG gives the LLM both the text and the connections.

What It Is

Use vector search to find relevant chunks, use the graph to find how the entities in those chunks are connected, and send both to the LLM.

Why It Is Required

Graphs are excellent at structure but lose the nuance of the original text. Vectors keep the text but lose the structure. Most real questions need a bit of both.

How It Works

Step	What Happens	Gives You
1	Vector search finds the chunks that answer the "what / why" part	Text details
2	Find the entities (nodes) mentioned in those chunks	Starting nodes
3	Walk the graph from those nodes	Connections: owner, manager, dependencies
4	Send text + graph path together to the LLM	Complete answer

The full worked example (the Orion delay question) is in **1.3, Part B**.

Practical note: Several graph databases (Neo4j among them) support vector indexes on node properties, so both searches can run in a single database. Otherwise, keep a shared `chunk_id` in both the vector store and the graph to join them.

1.11 Choosing a Graph Database

Database	Query Language	Notes
Neo4j	Cypher	Most widely used; large ecosystem; built-in vector index; strong LangChain / LlamaIndex support
Amazon Neptune	Gremlin, openCypher, SPARQL	Managed on AWS; fits teams already on AWS

Database	Query Language	Notes
Memgraph	Cypher	In-memory, very fast for real-time traversals
FalkorDB	Cypher	Lightweight, low-latency, designed with GraphRAG workloads in mind
ArangoDB	AQL	Multi-model — graph, document and key-value in one
NebulaGraph	nGQL	Built for very large graphs distributed across machines

Starting recommendation: Neo4j, because most Graph RAG tutorials, integrations and Text-to-Cypher examples target Cypher. Check the current feature set and licensing of any database before committing — these products evolve quickly.

1.12 When NOT to Use a Graph

Graph RAG is powerful and fashionable. That makes it easy to over-use.

Situation	Why a Graph Is the Wrong Choice
Most questions are "What does the policy say about X?"	Single-hop text lookup — vector RAG is simpler and better
Data has few shared entities	The graph ends up as disconnected islands
Questions are mostly numeric aggregations	SQL is purpose-built for this
Small team, tight deadline	Extraction, schema design and entity resolution are real ongoing work
Documents change daily	Re-extraction and re-resolution on every change is costly

> **Rule:** Add a graph when you can list at least five real user questions that need 2+ hops. If you cannot, you do not need a graph yet.

SECTION 2 — WHERE DOES THE DATA LIVE? VECTOR DB vs GRAPH DB vs SQL

Why This Section Is Required

Section 1 showed *why* a graph database is needed. Now one practical question must be crystal clear: **when a document enters a RAG system, what exactly gets saved, and where?** A common confusion is thinking a graph database is "another vector database". It is not. Each store saves a completely different *shape* of information, and each is good at a completely different kind of question.

Simple analogy: Think of a company's filing system.

Store	Office Equivalent	What It Holds
Vector database	The document library, organised by topic	The actual paragraphs of text, shelved so that similar topics sit together
Graph database	The whiteboard with the org chart and project map	Names in boxes, arrows between them — who connects to what
SQL database	The Excel register kept by HR and Finance	Rows and columns of exact values — dates, counts, amounts

A question like "What does the leave policy say?" goes to the library. "Who does Karan report to?" goes to the whiteboard. "How many people joined in Q3?" goes to the register.

2.1 The Three Stores at a Glance

Aspect	Vector Database	Graph Database	SQL Database
Basic unit stored	Chunk (a piece of text) + its embedding	Node (entity) and edge (relationship)	Row in a table
What it represents	Meaning of a passage	Entities and how they connect	Exact structured facts
Stores the original text?	Yes — the chunk text	Usually no — only short facts; links back to chunks	No — only field values
Stores embeddings?	Yes — always	Optional (some support vector indexes)	Optional (e.g., pgvector)
How you search	Similarity: "find text that means something like this"	Traversal: "follow these arrows"	Exact query: filter, count, sum, join
Query language	Vector similarity + metadata filters	Cypher, Gremlin, SPARQL	SQL
Best question type	"What does the policy say about X?"	"Who / what is connected to X, and how?"	"How many / how much / list all where…"
Weak at	Relationships, counting, exact values	Long free-text explanation	Meaning, fuzzy wording

Aspect	Vector Database	Graph Database	SQL Database
Built by	Chunking + embedding model	LLM entity/relationship extraction, or structured sources	Loading tables / spreadsheets / app databases
Examples	Pinecone, Weaviate, Qdrant, Milvus, Chroma, pgvector	Neo4j, Amazon Neptune, Memgraph, FalkorDB	PostgreSQL, MySQL, SQL Server

---

2.2 What Gets Stored in the Vector Database

Each record in a vector database has **three parts**:

Part	What It Is	Example
ID	Unique chunk identifier	`hr-policy_2025-04_p12_c3`
Vector (embedding)	A list of numbers representing the meaning of the chunk (e.g., 768 or 1536 numbers)	`[0.021, -0.113, 0.487, … ]`
Payload	The chunk text + metadata	Text, source file, page, section, access level, version

Example record:


```
{
  "id": "hr-policy_2025-04_p12_c3",
  "vector": [0.021, -0.113, 0.487, "... 1533 more numbers ..."],
  "payload": {
    "text": "This is from the Leave Policy section of the TechNova HR Policy. Sick leave is 12 days per year. Unused sick leave
lapses at year end.",
    "doc_id": "hr-policy",
    "doc_version": "2025-04",
    "section": "3. Leave Policy",
    "page": 12,
    "element_type": "text",
```

```
    "access_level": "all_employees"
  }
}
```

What typically goes into the vector database:

Content	Stored As
Policy paragraphs, manuals, FAQs	Text chunks + embeddings
Small tables	Markdown table chunk, or row sentences (Section 3.2)
Large tables	A summary of the table (full table kept in doc store or SQL)
Images, charts	Vision-LLM description as text, or image embedding (Section 3.3)
Audio / video	Transcript chunks with timestamps
Spreadsheets	Only a *description* of each sheet — the rows go to SQL
Graph community summaries	Summary text + embedding (Section 1.9)

2.3 What Gets Stored in the Graph Database

A graph database stores **nodes**, **edges** and **properties** — *not* paragraphs of text.

The nodes, edges and properties themselves are explained in **Section 1.2**.

Key point: The graph stores the **facts and connections** extracted from the text. The full explanatory text stays in the vector database. The `MENTIONED_IN` edges connect the two.

What typically goes into the graph database:

Content	Stored As
People, teams, departments	Nodes
Reporting lines, memberships	Edges

Content	Stored As
Projects, products, services, systems	Nodes
Dependencies, ownership, usage	Edges
Regulations, clauses, controls	Nodes + `REQUIRES` / `SATISFIES` edges
Pointers to source text	`Chunk` nodes + `MENTIONED_IN` edges
Community summaries (GraphRAG)	`Community` nodes with a summary property

---

2.4 What Gets Stored in the SQL Database

Content	Example Table	Example Columns
Employee records	`employees`	emp_id, name, department, level, join_date, location
Salary bands	`salary_bands`	level, title, min_salary, max_salary
Attrition figures	`dept_stats`	department, fiscal_year, attrition_pct
Spreadsheets uploaded by users	One table per sheet	As in the sheet
RAG bookkeeping	`indexed_documents` , `indexed_chunks`	Record manager (Section 4.7)

SQL is where anything needing **exact numbers, counting, sorting or arithmetic** belongs.

2.5 One Document, Three Stores — Worked Example

Input: One page of the TechNova "Engineering Organisation & Leave" document:

> *Karan Bhatia leads the Platform team, which owns Project Orion. Karan reports directly to Priya Sharma, the CTO. Orion's backend is written in Python and Go.*
>
> *Sick leave is 12 days per year. Unused sick leave lapses at year end.*

> *Table: Leave Entitlement — Earned 18, Sick 12, Casual 6.*

What each store receives from this one page:

Piece of Content	Vector Database	Graph Database	SQL Database
"Karan leads Platform… reports to Priya…"	Chunk text + embedding	Nodes: Karan, Platform, Orion, Priya. Edges: LEADS, OWNS, REPORTS_TO	—
"Orion's backend is Python and Go"	Chunk text + embedding	Nodes: Python, Go. Edges: Orion USES Python, Orion USES Go	—
"Sick leave is 12 days… lapses at year end"	Chunk text + embedding	Optional: Policy node only	—
Leave entitlement table	Markdown table chunk + embedding	—	Rows in `leave_entitlement` table
Page and section info	Metadata on each chunk	`Chunk` nodes with chunk_id	—
Document hash, version	—	—	Row in `indexed_documents`

What each store is then able to answer:

Question	Answered By	Why
"What happens to unused sick leave?"	Vector DB	Explanatory text lookup
"Which technologies do projects under the CTO use?"	Graph DB	4-hop traversal
"What is the total leave days across all types?"	SQL	Arithmetic: 18 + 12 + 6 = 36
"Why is Orion delayed and who do I escalate to?"	Vector DB + Graph DB	Reason is in text; escalation path is in the graph

---

2.6 How the Stores Are Linked

The stores are only useful together if they share identifiers.

Shared Key	Lives In	Used To
`chunk_id`	Vector DB record ID, graph `Chunk` node, record manager	Jump from a graph entity to the original text (and back)
`doc_id` + `doc_version`	All three stores	Update or delete a document everywhere at once
`employee_id` , `project_id`	Graph node property, SQL primary key	Join relationship answers with exact numbers
`access_level` / `allowed_groups`	Vector metadata, graph property, SQL row-level policy	Enforce the same permissions in every store

Rule: When a document is updated or deleted (Section 4.6), it must be updated or deleted in **all** stores. Otherwise the graph points to text that no longer exists, or the vector DB says one thing while the graph says another.

2.7 Which Store Answers Which Question

Question Pattern	Store	Example
"What does … say about …?"	Vector	"What does the policy say about work from home?"
"Explain / summarise …"	Vector (+ RAPTOR / GraphRAG summaries)	"Summarise the performance review process."
"Who reports to / owns / depends on …?"	Graph	"Who reports to the CTO?"
"How is A connected to B?"	Graph	"How is Project Orion connected to the CEO?"
"What breaks if … goes down?"	Graph	"What depends on Redis Cluster 2?"
"How many / total / average …?"	SQL	"How many joined in Q3 2024?"
"List all … where …"	SQL	"List L3 engineers in Pune."
"Why … and who …?"	Vector + Graph	"Why is Orion delayed and who owns it?"
"Is our X above the industry average?"	SQL + Web	"Is Data & AI attrition above average?"

> **Remember:** A vector database stores **what documents say**. A graph database stores **how things are connected**. A SQL database stores **exact values**. Production RAG systems usually use all three, and a router (Section 12, Section 4) decides which one each question needs.

SECTION 3 — MULTIMODAL INGESTION: TABLES, IMAGES AND EVERYTHING ELSE

Why This Section Is Required

Real company knowledge is not clean paragraphs. It is PDFs with tables, slide decks full of charts, scanned forms, Excel sheets, whiteboard photos, recorded town halls, and code. Basic RAG tutorials use a single `PyPDFLoader` and assume text comes out. In reality, a large share of the most important facts — numbers in tables, trends in charts, processes in diagrams — never make it into the vector database at all.

Simple analogy: Imagine translating a cookbook for a friend, but you only translate the paragraphs and skip every ingredient table and every photo. Your friend now has a cookbook that describes the dish beautifully but has no quantities and no idea what it should look like. That is what naive ingestion does to your documents.

What it is: Techniques for extracting and representing tables, images, scans, spreadsheets, audio, video and code so that they can be retrieved accurately from a vector database.

> **The golden rule of multimodal RAG:** Everything is eventually either **converted to text** (and embedded as text) or **embedded directly** with a model that understands that modality. Every technique in this section is one of those two choices.

3.1 Document Parsing — The Step Everyone Skips

Simple Analogy

Before cooking, you sort groceries: vegetables here, spices there, dairy in the fridge. Parsing sorts a document into its element types — heading, paragraph, table, image, caption, footer — so each can be handled the right way.

What It Is

Layout-aware parsing detects the **structure** of a page — not just the text on it — and outputs typed elements.

```
Naive extraction of a PDF page:
  "TechNova HR Policy 2024 Page 12 3. Leave Policy Leave Type Days
   Carry Forward Earned 18 Yes, up to 45 Sick 10 No Figure 2: Leave
   approval flow Confidential"

Layout-aware parsing of the same page:
```

```
[Header]    "TechNova HR Policy 2024"          → discard
[Title]     "3. Leave Policy"                  → use as metadata
[Table]     Leave Type | Days | Carry Forward  → table pipeline
[Image]     (flowchart)                        → image pipeline
[Caption]   "Figure 2: Leave approval flow"    → attach to image
[Footer]    "Confidential"                     → discard
[PageNum]   "12"                               → use as metadata
```

Common Parsing Tools

Tool	Type	Strength
PyMuPDF / pdfplumber	Open source, rule-based	Fast; good for digital PDFs with simple layouts
Unstructured	Open source + hosted	Many file types; returns typed elements
Docling (IBM)	Open source	Strong table structure recognition; exports clean Markdown
LlamaParse	Hosted	LLM-assisted parsing of complex PDFs
Azure Document Intelligence / AWS Textract / Google Document AI	Cloud services	OCR, forms, tables at enterprise scale
Vision LLMs (send the page as an image)	Model-based	Handles messy layouts nothing else can; most expensive

Why It Matters

Headers and footers repeated on every page ("Confidential — TechNova 2024") become noise in every chunk. Removing them alone noticeably sharpens embeddings. Section titles captured during parsing become free metadata for self-query retrieval (Section 12, 2.1).

3.2 Tables

Simple Analogy

A train timetable is useless as a paragraph: "Train 12627 departs 21:15 arrives 13:30 Train 12649 departs 22:00…" Nobody can read that. The table *is* the meaning — rows and columns carry it. Table RAG is about keeping that structure alive.

The Problem

Tables hold the highest-value facts in most business documents — salaries, leave days, prices, targets, specifications. They are also the thing naive extraction destroys most completely (see Failure 3). There are four good strategies, and the right one depends on the table.

Strategy 1: Keep the Table as a Markdown Table

What: Store the whole table as one chunk, written as a Markdown table. LLMs read Markdown tables easily.

Example table:

Leave Type	Days per Year	Carry Forward
Earned	18	Yes, up to 45
Sick	12	No
Casual	6	No

Rules:

- **Never cut a table into two chunks.** One table = one chunk.
- **Add a label on top of the table** before storing it. This is explained step by step below.

Best for: Small and medium tables (under ~30 rows).

Why a Table Needs a Label on Top

The problem: A table is mostly numbers and short words. On its own, it does not say *what* it is about.

Look at the salary table alone:

Level	Title	Min (Rs)	Max (Rs)
L1	Associate Engineer	4,00,000	6,50,000
L3	Senior Engineer	12,00,000	18,00,000
L8	VP	70,00,000	1,00,00,000

The words "salary", "TechNova", "pay" or "compensation" are **not in the table**. So when a user asks *"What is the salary for a senior engineer?"*, this chunk may not be found.

The fix: Before storing, write 1–2 lines on top of the table saying what it is. (This is called *adding context*. In Section 12 it was called *contextual retrieval*.)

Simple analogy: A box of files with no label on the outside. Nobody knows what is inside until they open it. Write "Salary Bands 2025 — HR" on the box, and anyone can find it.

How to Build the Label — Step by Step

Step	What to Take	Where It Comes From	Example
1	Document name	File name or document title	"TechNova Compensation Policy 2025"
2	Section heading	The heading just above the table	"Section 4: Salary Structure"
3	Table caption	The line above or below the table	"Table 4: Salary Bands by Level"
4	One-line meaning (optional)	Write it yourself, or ask an LLM	"Yearly salary range (minimum and maximum) for each job level."
5	Join them and put them **on top** of the table	—	See the output below

Output — the chunk that is actually stored:

> **Document:** TechNova Compensation Policy 2025
> **Section:** 4. Salary Structure

> **Table:** Salary Bands by Level — yearly salary range (minimum and maximum) for each job level.
>
> | Level | Title | Min (Rs) | Max (Rs) |
> |---|---|---|---|
> | L1 | Associate Engineer | 4,00,000 | 6,50,000 |
> | L3 | Senior Engineer | 12,00,000 | 18,00,000 |
> | L8 | VP | 70,00,000 | 1,00,00,000 |

Before vs after:

	Chunk Contains	Question: "Salary for a senior engineer?"
Without label	Only the table	Weak match — may not be found
With label	"Salary", "Compensation", "yearly", + the table	Strong match — found at the top

If the table has no heading or caption: Ask an LLM to write the label.

> *Prompt:* "Here is a table from TechNova's Compensation Policy. Write one line saying what this table shows."
> *Output:* "Yearly salary range for each job level at TechNova."

Strategy 2: One Row = One Sentence

What: Turn every row into a full sentence. Each sentence repeats the column names, so it makes sense on its own.

Input table:

Level	Title	Min (Rs)	Max (Rs)
L1	Associate Engineer	4,00,000	6,50,000
L3	Senior Engineer	12,00,000	18,00,000
L8	VP	70,00,000	1,00,00,000

Steps:

Step	What to Do	Example
1	Read the column names	Level, Title, Min, Max
2	Take one row	L3, Senior Engineer, 12,00,000, 18,00,000
3	Fill a simple sentence template	"{Title} ({Level}): salary from Rs {Min} to Rs {Max} per year."
4	Add the label from above (document + table name)	"TechNova Salary Bands — …"
5	Store each sentence as its own chunk	3 rows → 3 chunks

Output — the chunks that are stored:

Chunk	Text
1	"TechNova Salary Bands 2025 — Associate Engineer (L1): salary from Rs 4 lakh to Rs 6.5 lakh per year."
2	"TechNova Salary Bands 2025 — Senior Engineer (L3): salary from Rs 12 lakh to Rs 18 lakh per year."
3	"TechNova Salary Bands 2025 — VP (L8): salary from Rs 70 lakh to Rs 1 crore per year."

Why it works: The question "What is the salary for a senior engineer?" now matches Chunk 2 almost word for word.

Best for: Lookup tables, where each question is about one row.

Strategy 3: Summary for Retrieval, Full Table for the LLM (Multi-Representation)

What: Section 12's multi-representation indexing (1.3), applied to tables. An LLM writes a description of the table — what it contains and its key facts. The description is embedded. When it matches, the full table is sent to the LLM.

```
Embedded (for retrieval):
  "Table of TechNova department-wise attrition for FY 2024-25.
   Covers 8 departments. Overall attrition 11.2%. Highest: Data & AI
   (14.7%) and Sales (14.4%). Lowest: HR & Admin (9.7%)."
```

```
Stored in document store (sent to LLM):
  The full 8-row attrition table in Markdown.
```

Best for: Large tables, and questions that compare across rows ("Which department had the highest attrition?").

Strategy 4: Send the Table to SQL (Don't Embed It at All)

What: Load the table into a SQL database and answer questions with Text-to-SQL (Section 12, 2.2). Keep only a table *description* in the vector store so the router knows the table exists.

```
User asks: "What is the average max salary across L1 to L5?"

Embedding cannot do arithmetic. An LLM reading a 40-row table
might miscalculate. SQL computes it exactly:

SELECT AVG(max_salary) FROM salary_bands
WHERE level IN ('L1','L2','L3','L4','L5');
```

Best for: Large numeric tables and any question involving sums, averages, filters, sorting or counting.

Choosing a Table Strategy

Table Characteristic	Best Strategy
Small (under ~30 rows), mixed text and numbers	1 — Markdown serialisation
Lookup table, one row per question	2 — Row-level chunks
Large table, comparative questions	3 — Summary + full table
Numeric, needs calculation	4 — SQL
Critical, high-traffic table	Combine 2 or 3 with 4

3.3 Images, Charts and Diagrams

Simple Analogy

Describing a photo to a friend over the phone. You cannot send the photo, so you describe it in words: "It's a bar chart, Q4 is the tallest bar, about double Q1." Your friend can now answer questions about it. Image captioning does this for your RAG system. Alternatively, you could give your friend eyes — that is a multimodal embedding.

What Kinds of Images Appear in Documents

Image Type	What the Information Is	Example
Charts and graphs	Numbers and trends	Quarterly revenue bar chart
Diagrams and flowcharts	Process and structure	Leave approval flow
Org charts	Relationships	Reporting hierarchy
Screenshots	UI steps, error messages	"How to apply in HRConnect"
Photos	Physical objects, scenes	Office layout, product photo
Scanned text	Text trapped in pixels	Signed offer letter

There are three approaches.

Approach 1: Caption / Describe With a Vision LLM → Embed the Text

What: Send each image to a vision-capable LLM with a prompt tuned to the image type. Store the description as a text chunk, linked to the original image.

Step-by-step with example output: **3.9 E**.

Why it works: The chart is now searchable by meaning ("which quarter had highest revenue?"). And at answer time, you can show the user the original image alongside the answer.

Tip for diagrams and org charts: Ask the vision LLM to output the diagram as structured data (a list of steps, or `A -> B` edges). An org chart described as edges can be loaded directly into a graph (Section 1).

Approach 2: Multimodal Embeddings (Images and Text in the Same Space)

What: Use an embedding model trained on image–text pairs (for example, CLIP-family models). A photo of a laptop and the words "a laptop" land close together in the same vector space. You embed images directly — no captioning.

```
Indexing:  image → multimodal embedding model → vector
Querying:  "photo of the Bengaluru office reception"
           → same model → vector → nearest images
```

Best for: Natural photos, product images, visual search ("find slides that look like this").

Limitation: General image embeddings are weak on charts, tables and dense text inside images. For documents, Approach 1 or 3 is usually better.

Approach 3: Page-Image Retrieval (ColPali Style)

What: Skip parsing entirely. Render every PDF page as an image and embed the whole page with a vision-language model. ColPali applies ColBERT's late-interaction idea (Section 12, 1.6) to page images — many vectors per page, one per image patch.

```
Indexing:   each page → image → vision model → multi-vector embedding
Querying:   question → query vectors → matched against page patches
Answering:  top 2–3 page images → sent to a vision LLM → answer
```

Why it matters: Tables, charts, layout and text are all preserved exactly as the author designed them — nothing is lost in extraction. It is particularly strong on visually rich documents: slide decks, brochures, infographics, financial reports.

Trade-off: More storage (many vectors per page), and answering requires a vision LLM reading page images — slower and more expensive than text. Retrieval is at page level, not chunk level.

Choosing an Image Strategy

Situation	Best Approach
Charts and diagrams inside mostly-text documents	1 — Vision LLM captioning
Product photos, visual similarity search	2 — Multimodal embeddings
Slide decks, infographics, complex visual reports	3 — Page-image retrieval
Org charts and flowcharts	1, outputting structured edges → load into graph

---

3.4 Scanned Documents and OCR

Simple Analogy

A photocopy of a letter looks like text to you, but to a computer it is just a picture of ink. OCR (Optical Character Recognition) is the process of "reading" that picture and turning it back into typed text.

What It Is

Scanned PDFs, phone photos of forms and faxed documents contain no extractable text layer. OCR converts them to text before any chunking can happen.

Tools and step-by-step process: **3.9 F**.

Indian Context Tip

Many Indian enterprise documents mix English with Hindi, Kannada, Tamil or other scripts — for example, government forms and vendor invoices. Test your OCR on real samples in every script you expect. Engines vary widely in Indic-script accuracy.

Quality Guard

OCR makes mistakes: "10 days" can become "1O days", "Rs 434" can become "Rs 43A". Store an OCR confidence score as metadata, and flag low-confidence chunks — do not let the LLM quote numbers from them without a warning.

3.5 Spreadsheets and CSV Files

Simple Analogy

A spreadsheet is already a database that forgot it was one. The worst thing you can do is flatten it into paragraphs. The best thing is to let it be a database again.

The Problem

A 5,000-row Excel sheet of employee records turned into text chunks produces 5,000 near-identical embeddings. Semantic search over them is nearly meaningless — and counting or averaging over retrieved chunks is impossible.

Recommended Approach

Load the rows into **SQL**. Store only a short **description** of each sheet in the vector database, so the router knows the sheet exists and sends number questions to Text-to-SQL. Step by step: **3.9 G**.

Exception: Small spreadsheets that are really *documents* (a two-column FAQ sheet, a glossary) can be row-chunked like a table (Strategy 2 in 3.2).

3.6 Audio and Video

Simple Analogy

You cannot search inside a recording by listening to all of it. But a transcript with timestamps is like a book with page numbers — you can search it, and jump straight to minute 34.

What It Is

Convert speech to text **with timestamps** and **speaker names**. Add descriptions of what is shown on screen. Then the recording can be searched like a document, and the answer can link to the exact minute. Step by step: **3.9 I**.

3.7 Code and Technical Documentation

Simple Analogy

You would never cut a recipe in the middle of step 3 and file the second half under a different dish. Code is the same — a function cut in half is meaningless.

Rules

Rule	Why
Split on syntax boundaries — functions, classes, methods	A half-function cannot be understood
Keep the file path, class name and function signature with every chunk	"def calculate()" alone is ambiguous across 40 files
Add an LLM-generated docstring-style summary for retrieval	Users ask in English; code is written in Python — the summary bridges the gap (multi-representation)
Index README, API docs and code together, linked by module	Questions often need both "how" (docs) and "what exactly" (code)
Consider a code graph	Function A CALLS function B, module X IMPORTS module Y — a natural graph use case

Tools and step by step: **3.9 J**.

3.8 The Unified Ingestion Pipeline

Simple Analogy

A post office sorting centre. Every parcel comes through one door, is scanned, sorted to the right conveyor by type, processed, labelled with a tracking number, and delivered to the right shelf. A good ingestion pipeline is that sorting centre.

The Full Pipeline

Stage by stage:

Stage	What Happens	Output
1. Deduplication check	File hash and content hash compared with the record manager (Section 4)	Duplicate → stop. New or updated → continue
2. Detect file type	Identify PDF, DOCX, scan, spreadsheet, image, audio/video, code	Route to the right processor
3. Process by type	Run the processor for that type (see 3.9)	Text chunks, tables, image descriptions, transcripts, SQL tables
4. Enrich	Add contextual prefix, metadata, PII detection	Chunks ready for storage
5. Embed and store	Send each piece to the right store (Section 2)	Vector DB, doc store, SQL, graph
6. Register	Record every document and chunk in the record manager (Section 4.7)	Enables dedup, updates and deletes later

The Metadata Every Chunk Should Carry

Every chunk carries identity, location, type, time and security fields — full list in **Section 5.1**. The most important ones: `content_hash` (deduplication), `doc_version` (versioning), `access_level` (security), `section` and `page` (citations), `element_type` (how to display the result).

3.9 Step by Step: Extracting and Storing Each Data Type

Simple Analogy

A courier company receives letters, parcels, cold-chain medicines and furniture. Each is unpacked differently, but every item ends up with the same thing: a **label**, a **tracking number** and a **shelf**. In RAG, every data type is extracted differently, but everything ends up as the same thing: a **vector + text + metadata** record in the vector database.

Every data type goes through the same pipeline (3.8). Only the **Extract** step changes. This section shows that step for each type.

Quick Reference: What Becomes the Embedding for Each Type

Data Type	Main Extraction Technique	What Gets Embedded	What Is Sent to the LLM	Stored In
Digital PDF (text)	Text-layer extraction + layout parsing	Text chunk	Text chunk	Vector DB

Data Type	Main Extraction Technique	What Gets Embedded	What Is Sent to the LLM	Stored In
Word (DOCX)	Read paragraphs, headings, tables from the file structure	Text chunk	Text chunk	Vector DB
Web page (HTML)	Strip menus and ads, keep main content	Text chunk	Text chunk	Vector DB
Table	Table structure detection	Row sentence or table summary	Full table (Markdown)	Vector DB + doc store / SQL
Image / chart / diagram	Vision LLM description	Description text	Description (+ original image)	Vector DB + image file storage
Scanned document	OCR	OCR text chunk	OCR text chunk	Vector DB
Excel / CSV	Load as a data table	Sheet description	SQL query result	SQL + description in Vector DB
PowerPoint	Slide text + speaker notes + slide images	Slide text + visual description	Slide content (+ slide image)	Vector DB + slide images
Audio / video	Speech-to-text + key frames	Transcript chunk	Transcript chunk (+ timestamp link)	Vector DB (with timestamps)
Code	Syntax-aware splitting	Code summary (+ code)	Actual code	Vector DB (+ optional code graph)
Text with many people / teams / links	LLM entity and relationship extraction	Text chunk	Text chunk + graph path	Graph DB + Vector DB

A. Digital PDFs (Text-Based)

A "digital" PDF was created from software (Word, Google Docs), so the text is already inside the file. You can select and copy it.

Extraction techniques

Technique / Tool	How It Works	Best For
Basic text extraction (pypdf)	Reads the text layer page by page	Simple, single-column documents

Technique / Tool	How It Works	Best For
Position-aware extraction (PyMuPDF, pdfplumber)	Reads text *with* its position, font size and bold	Detecting headings, removing headers and footers
Layout-aware parsing (Docling, Unstructured)	Detects titles, paragraphs, lists, tables, images as separate elements	Complex layouts, two columns, mixed content

Step by step

Step	What You Do	Example Output
1	Open the PDF and read each page	25 pages of raw text
2	Detect element types (title, paragraph, table, image, header, footer)	`[Title] 3. Leave Policy`, `[Paragraph] Sick leave is 12 days…`
3	Remove noise — repeated headers, footers, page numbers	"Confidential — TechNova" removed from every page
4	Send tables and images to their own pipelines (C and D below)	Tables and images handled separately
5	Chunk the paragraphs by section (structure-aware or semantic chunking)	84 chunks
6	Add a short context line at the start of each chunk	"From TechNova HR Policy, Section 3: Leave Policy."
7	Attach metadata	source, page, section, doc_version, access_level
8	Embed each chunk and store it	84 records in the vector database

What one stored record looks like: see Section 2.2.

B. Word Documents (DOCX)

A DOCX file is already structured internally — headings, paragraphs and tables are stored as separate labelled parts. This makes it *easier* than PDF.

Extraction techniques

Technique / Tool	How It Works	Best For
python-docx	Reads each paragraph with its style name ("Heading 1", "Normal") and each table	Full control
Convert to Markdown (e.g., Docling, Pandoc, Mammoth)	Turns headings into `#`, tables into Markdown tables	Clean input for Markdown-aware chunking
Unstructured	Returns typed elements, like for PDF	Mixed file collections

Step by step

Step	What You Do	Example Output
1	Read paragraphs in order, with their style	`Heading 1: Leave Policy`, `Normal: Sick leave is…`
2	Use headings to build the section path	"HR Policy > Leave Policy > Sick Leave"
3	Pull out tables separately	Table → table pipeline (C)
4	Pull out embedded images	Image → image pipeline (D)
5	Split text at headings (one section = one or more chunks)	Chunks that never cross a section boundary
6	Add section path + metadata to each chunk	`section: "Leave Policy > Sick Leave"`
7	Embed and store	Records in the vector database

Tip: Also capture tracked changes and comments only if they matter. Usually you want the final accepted text only.

C. Web Pages (HTML)

Extraction techniques

Technique / Tool	How It Works	Best For
Main-content extraction (trafilatura, Readability)	Removes menus, footers, ads and cookie banners; keeps the article	News, blogs, help articles

Technique / Tool	How It Works	Best For
HTML parsing (BeautifulSoup)	You choose exactly which tags to keep (`<article>` , `<h2>` , `<table>`)	Known site layouts, internal wikis
Headless browser (Playwright)	Loads the page like a real browser, so JavaScript content appears	Pages that build content with JavaScript

Step by step

Step	What You Do	Example Output
1	Fetch the page (or read the saved HTML)	Raw HTML
2	Remove boilerplate — navigation, footer, ads, scripts	Main content only
3	Convert to Markdown, keeping `<h1>` – `<h3>` as headings and `<table>` as tables	Clean structured text
4	Chunk by headings	One chunk per sub-topic
5	Add metadata: URL, page title, last-updated date	`url: "intranet/hr/leave"`
6	Embed and store	Records in the vector database

D. Tables Inside Documents

(Strategies explained in 3.2 — this is the extraction and storage walkthrough.)

Extraction techniques

Technique / Tool	How It Works	Best For
Rule-based table extraction (pdfplumber, Camelot)	Uses ruling lines and text alignment to find rows and columns	Clean digital PDFs with visible borders
AI table-structure models (Docling, Table Transformer)	A model detects the table region, rows, columns and merged cells	Tables without borders, complex headers

Technique / Tool	How It Works	Best For
Cloud document AI (Azure Document Intelligence, AWS Textract, Google Document AI)	Managed OCR + table detection	Scanned tables, large volumes
Vision LLM	Send the page image; ask for the table as Markdown or JSON	Very messy tables nothing else can read

Step by step (salary band table example)

Step	What You Do	Example Output
1	Detect the table region on the page	Table found on page 18
2	Extract rows and columns into a structured grid	8 rows × 4 columns
3	Clean it — fix merged header cells, remove empty rows, keep units ("Rs")	Clean table
4	Save the full table as Markdown in a document store (and in SQL if it is numeric)	`salary_bands` table in SQL
5	Create the retrievable text: either one sentence per row, or one summary of the table	"Band L3 (Senior Engineer): Rs 12 lakh to Rs 18 lakh per year."
6	Add the section title and table caption as context	"From TechNova Compensation Policy, Table 4: Salary Bands."
7	Embed the row sentences or summary; store with a pointer to the full table	`table_id: "comp_policy_t4"`

Steps 5–6 are explained with examples in **3.2** (label on top, one row = one sentence).

E. Images, Charts and Diagrams

Extraction techniques

Technique / Tool	How It Works	Best For
Extract embedded images (PyMuPDF, python-docx)	Pulls the image files out of the document, with page number and caption	First step for every document

Technique / Tool	How It Works	Best For
Vision LLM captioning	The model "looks" at the image and writes a detailed description	Charts, diagrams, screenshots
Chart-to-data	Vision LLM asked to output the chart's numbers as a table	Bar and line charts
Diagram-to-edges	Vision LLM asked to output "A → B" steps	Flowcharts, org charts (can load into a graph)
OCR on the image	Reads text written inside the image	Screenshots with error messages
Multimodal embedding (CLIP-family models)	Embeds the image itself, with no text	Photos, visual similarity search

Step by step (revenue chart example)

Step	What You Do	Example Output
1	Extract the image and its caption from page 7	`p7_fig3.png` , caption "Figure 3: Quarterly Revenue"
2	Filter out useless images — logos, icons, decorative lines (by size or a quick check)	Logo on every page skipped
3	Save the original image in file storage	`s3://docs/annual_report/p7_fig3.png`
4	Send it to a vision LLM with a type-specific prompt	"Describe this chart: type, axes, every value, main trend."
5	Receive the description	"Bar chart of quarterly revenue FY 2024-25 in Rs crore: Q1 98, Q2 104, Q3 111, Q4 121…"
6	Add context — document name, page, caption, nearby paragraph	"From TechNova Annual Report, page 7, Figure 3."
7	Embed the description; store it with a link to the original image	Record with `image_path` in metadata
8	At answer time, show the image next to the answer	User sees the real chart

One stored record


```
{
  "id": "annual-report_p7_fig3",
  "text": "From TechNova Annual Report FY 2024-25, page 7, Figure 3: Quarterly Revenue. Bar chart in Rs crore: Q1 98, Q2 104, Q3
```

```
111, Q4 121. Revenue grew every quarter; Q4 highest. Total Rs 434 crore.",
  "metadata": {"element_type": "image", "image_path": "s3://docs/annual_report/p7_fig3.png", "page": 7}
}
```

F. Scanned Documents and Photos of Documents (OCR)

Extraction techniques

Technique / Tool	How It Works	Best For
Tesseract	Classic open-source OCR	Clean printed pages
PaddleOCR / EasyOCR	Deep-learning OCR, many languages	Mixed fonts, tilted text, Indic scripts
Cloud OCR (Azure, AWS Textract, Google Document AI)	Managed OCR with forms, tables and handwriting	Forms, invoices, large volumes
Vision LLM	Reads the page image with understanding of context	Messy handwriting, stamps, unusual layouts

Step by step (signed offer letter example)

Step	What You Do	Example Output
1	Detect that the page has no text layer	Page 1 returns empty text → scanned
2	Convert the page to a high-resolution image (around 300 DPI)	`offer_p1.png`
3	Clean the image — straighten tilt, remove shadows, increase contrast	Sharper image
4	Run OCR — get text, its position and a confidence score	"CTC: Rs 18,00,000 per annum" (confidence 0.97)
5	Fix common OCR errors with rules or an LLM (O→0, l→1 in numbers)	"1O days" → "10 days"
6	From here on, treat it like a digital PDF (A): layout, chunk, context	Chunks
7	Add metadata including `ocr_confidence`	Low-confidence chunks flagged
8	Embed and store	Records in the vector database

G. Excel and CSV Files

(Why: see 3.5 — the rows go to SQL, only the description goes to the vector database.)

Extraction techniques

Technique / Tool	How It Works	Best For
pandas	Reads each sheet into a DataFrame	Most spreadsheets
openpyxl	Reads cells, merged cells, formulas and sheet names	Messy sheets, multiple header rows
Load to SQL (pandas `to_sql`, database import)	Turns each sheet into a real table	Anything numeric or large

Step by step (Joiners_FY25.xlsx example)

Step	What You Do	Example Output
1	Read every sheet	2 sheets: "Joiners_FY25", "Summary"
2	Find the real header row; clean column names	`emp_id, name, department, join_date, level, location`
3	Fix data types — dates as dates, numbers as numbers	`join_date` = date
4	Load each sheet into SQL	Table `joiners_fy25` (212 rows)
5	Write a description of each table (an LLM can draft it)	"One row per employee who joined in FY 2024-25. Columns: … 212 rows."
6	Embed the description only; store the table name in metadata	Record with `sql_table: "joiners_fy25"`
7	At question time: the description matches → router sends the question to Text-to-SQL	"How many joined in Q3?" → 44

H. PowerPoint Slides (PPTX)

Extraction techniques

Technique / Tool	How It Works	Best For
python-pptx	Reads each slide's title, text boxes, tables and speaker notes	Text-heavy slides
Render slides to images + vision LLM	Converts each slide to an image and describes charts and diagrams	Visual slides
Page-image retrieval (ColPali style, see 3.3)	Embeds each slide image directly	Very visual decks, infographics

Step by step (town-hall deck example)

Step	What You Do	Example Output
1	Read slide title, body text and speaker notes	Slide 5: "FY26 Hiring Plan" + notes
2	Extract tables on slides → table pipeline (D)	Hiring by department table
3	Render slides that have charts or diagrams to images → vision LLM (E)	"Chart shows hiring target of 400: Engineering 220…"
4	Combine into **one chunk per slide**: title + text + notes + visual description	Complete slide chunk
5	Add metadata: deck name, slide number	`slide: 5`
6	Embed and store; keep the slide image for display	Record with `slide_image_path`

Why one chunk per slide: A slide is already a single idea. Splitting it loses meaning; merging slides mixes topics.

I. Audio and Video

Extraction techniques

Technique / Tool	How It Works	Best For
Speech-to-text (Whisper-family models, cloud speech services)	Converts speech to text with timestamps	All recordings
Speaker diarisation (e.g., pyannote, or built into cloud services)	Labels who is speaking when	Meetings, interviews, town halls

Technique / Tool	How It Works	Best For
Key-frame extraction (ffmpeg)	Takes a picture when the screen changes (e.g., new slide)	Screen-shared presentations
Vision LLM on key frames	Describes what is on screen	Slides, demos, whiteboards

Step by step (September town hall recording)

Step	What You Do	Example Output
1	Extract the audio track from the video	`townhall.wav`
2	Transcribe with timestamps	`[00:34:12] "Our FY26 hiring target is 400…"`
3	Add speaker labels	`[00:34:12] Arjun (CEO): "Our FY26 hiring target is 400…"`
4	Extract key frames when slides change; describe them with a vision LLM	Frame at 00:34:05: "Slide: FY26 Hiring Plan…"
5	Chunk the transcript by topic or by 1–2 minute windows, keeping speaker turns intact	Chunk 00:34:05–00:35:40
6	Attach the matching slide description to each chunk	Transcript + slide text together
7	Add metadata: file, start and end time, speaker	`start: "00:34:12", speaker: "Arjun"`
8	Embed and store; the answer can link to the exact minute	"Watch at 34:12"

J. Code and Technical Documentation

Extraction techniques

Technique / Tool	How It Works	Best For
Language-aware text splitters (in LangChain, LlamaIndex)	Split on functions and classes for common languages	Quick start

Technique / Tool	How It Works	Best For
Syntax-tree parsing (tree-sitter)	Understands code structure exactly — functions, classes, imports, calls	Large codebases
LLM summaries	Writes a plain-English description of each function	Bridging English questions to code

Step by step (payments service repository)

Step	What You Do	Example Output
1	Walk the repository; skip generated files, dependencies and binaries	340 source files
2	Parse each file into functions and classes	`calculate_refund()` in `payments/refunds.py`
3	Keep the file path, class name, signature and docstring with each piece	Full context
4	Ask an LLM for a one-paragraph summary of each function	"Calculates refund amount after deducting gateway fees…"
5	Embed the summary (and optionally the code); store the code as the text sent to the LLM	Record with `file_path`, `function`
6	Optionally store "calls" and "imports" relationships in a graph (Section 1)	`(calculate_refund)-[:CALLS]->(get_fee)`

The Final Step Is the Same for Every Type

Whatever the source, the last step always looks like this:

```
records = []
for chunk in chunks:                          # chunks from any pipeline above
    records.append({
        "id": chunk.id,                       # stable ID: doc + version + position
        "vector": embed(chunk.embed_text),    # what should be FOUND
        "text": chunk.llm_text,               # what the LLM should READ
        "metadata": chunk.metadata,           # source, page, type, access, version
```

```
    })

vector_db.upsert(records)                    # upsert = insert new, update existing
record_manager.register(records)             # remember what was stored (Section 4.7)
```

Notice `embed_text` vs `llm_text`: For tables, images and code, what you *search on* (a summary or description) is often different from what the LLM should *read* (the full table, the original image, the actual code). Keeping both is the key idea behind multimodal RAG.

SECTION 4 — DEDUPLICATION AND DOCUMENT VERSIONING

Why This Section Is Required

The same content often arrives more than once:

How It Happens	Example
Uploaded twice	HR uploads the policy, then a manager uploads it again
Renamed copy	`HR_Policy_2025 (1).pdf`, `HR Policy - final.pdf`
Different format	The PDF and the Word version of the same policy
Nightly re-sync	The same folder is indexed again every night
New edition	`HR_Policy_2024.pdf` is replaced by `HR_Policy_2025.pdf`

What goes wrong:

Problem	What the User Sees
Wasted retrieval slots	Top 4 results are 3 copies of the same chunk
Wasted money	Every copy is embedded and stored again
Skewed answers	Content that is copied more gets retrieved more
Contradicting answers	Old version says 10 days, new version says 12 days

Simple analogy: A good librarian checks the catalogue before putting a donated book on the shelf. If it is already there, the copy is not shelved again. When a new edition arrives, the old edition goes to the archive.

The Core Scenario — The Same PDF Is Uploaded Again

The Situation

When	What Happens
Monday	HR uploads `HR_Policy_2025.pdf`. It is split into 84 chunks and stored in the vector database.
Wednesday	A manager uploads the **same file** again as `HR_Policy_2025 (1).pdf`.

What should the system do? Let us see what happens **without** protection, then **how to stop it**, then **how to clean up** if duplicates already got in.

Part A — What Happens With NO Protection

Most basic RAG code does this on every upload:

```
chunks = split(load_pdf(file))
vector_db.add(texts=chunks)     # add() creates a NEW random ID for every chunk
```

Step by step:

Step	What Happens	Vector DB Size
1	Monday: 84 chunks stored with random IDs (`a81f…` , `93cd…` , …)	84
2	Wednesday: same file uploaded again	84
3	Parsed again → the **same** 84 chunks	84
4	`add()` gives them **new** random IDs → stored again	**168**

Step	What Happens	Vector DB Size
5	Every chunk now exists **twice**	168

Why the database allows this: A vector database does not check content. It only checks IDs. New random IDs = "new" data, so it happily stores the copy.

What the user sees — "What is the sick leave policy?"

Rank	Chunk	Source
1	"Sick leave is 12 days per year…"	HR_Policy_2025.pdf
2	"Sick leave is 12 days per year…"	HR_Policy_2025 (1).pdf — **copy**
3	"Medical certificate needed beyond 2 days…"	HR_Policy_2025.pdf
4	"Medical certificate needed beyond 2 days…"	HR_Policy_2025 (1).pdf — **copy**

The damage:

Problem	Effect
Top 4 has only 2 different facts	"Unused sick leave lapses at year end" is pushed out → incomplete answer
Storage and embedding cost doubled	You paid to embed 84 chunks again for nothing
It keeps growing	Uploaded 5 times → 5 copies of everything
Wrong ranking	Documents uploaded more often "win" more searches
Messy citations	The answer cites the same policy twice under different file names

---

Part B — How to Stop It: Two Layers of Protection

Layer 1 — The Gatekeeper: Check Before Ingesting

Before parsing or embedding anything, check whether this document is already stored.

Step	Check	Example	Result
1	Calculate the **file hash**	`HR_Policy_2025 (1).pdf` → `a3f9…7c21`	—
2	Is this file hash in the record manager?	Yes — same as Monday's `HR_Policy_2025.pdf`	**STOP**
3	(If no) Extract and clean the text; calculate the **content hash**	Catches PDF vs Word, or a re-exported copy	STOP if found
4	(If no) Is the `doc_id` already known?	Same document, new edition	Version update (4.6)
5	(If no) Is it a near-duplicate?	97% overlap with an existing document	Flag for review (4.3)
6	Otherwise	Genuinely new document	Ingest

Example output (upload log):

```
Upload: HR_Policy_2025 (1).pdf
  File hash:  a3f9…7c21
  Result:     DUPLICATE — identical to HR_Policy_2025.pdf (uploaded Monday by HR)
  Action:     Skipped. 0 chunks embedded. Vector DB size stays 84.
```

What the user uploading it sees:

> "This document is already in the knowledge base (uploaded on Monday by HR). No changes were made."

Layer 2 — The Safety Net: Fixed Chunk IDs + Upsert

The gatekeeper can be skipped by mistake (a bug, a manual script, a re-run of a job). So the storage step itself must also refuse to create copies.

The idea: Give every chunk an ID made **from its content**, not a random one.

```
chunk_id = doc_id + "::" + hash(chunk text)

"Sick leave is 12 days per year…"  →  hr-policy::77a1c9e2
```

Same text → same ID, every time. Then use **upsert** instead of add:

Operation	Meaning	If the ID Already Exists
`add()`	Always insert	Error, or a copy with a new random ID
`upsert()`	Insert or update	**Overwrites** the existing record — no copy

Example output — same file stored twice:

	Using `add()` with random IDs	Using `upsert()` with content-based IDs
After Monday	84 chunks	84 chunks
After Wednesday	**168 chunks**	**84 chunks**
Duplicates	84	0

```
import hashlib

def chunk_id(doc_id, text):
    return f"{doc_id}::{hashlib.sha256(text.strip().lower().encode()).hexdigest()[:16]}"

ids = [chunk_id("hr-policy", c) for c in chunks]
collection.upsert(ids=ids, documents=chunks, embeddings=embed(chunks))
```

Use both layers:

Layer	Stops Duplicates?	Saves Embedding Cost?
Gatekeeper only	Yes, if it never fails	Yes — stops before embedding
Upsert only	Yes	No — the file is still parsed and embedded again
Both	**Yes, even if one fails**	**Yes**

Part C — What to Do in Each Upload Case

Case	Example	Detected By	Action
Exact same file	`HR_Policy_2025 (1).pdf`	File hash	Skip; tell the uploader it already exists
Same content, different file	`HR_Policy_2025.docx`	Content hash	Skip; optionally record it as another source
Updated edition	`HR_Policy_2026.pdf` (same `doc_id`)	`doc_id` in record manager	Version update — replace changed chunks only (4.6)
Almost the same, unknown	Draft with one word changed	MinHash (4.3)	Flag: "Is this a new version of HR Policy?"
Shares some chunks with others	Onboarding guide with the same leave table	Chunk hash (4.5)	Store only the new chunks; link the shared ones
Genuinely new	`Travel_Policy.pdf`	Nothing matches	Ingest normally

Part D — Duplicates Already Got In: How to Clean Up

If a system ran without protection for months, the vector database already has copies. Clean it once, then turn on both layers.

Step	What You Do	Example
1	Read every chunk's text and metadata from the vector DB	5,000 chunks
2	Clean and hash each chunk's text	5,000 hashes
3	Group chunks with the same hash	1,900 groups have more than one chunk
4	In each group, **keep one** — the oldest, or the one from the official source	Keep `HR_Policy_2025.pdf`
5	**Delete the rest**	2,300 copies deleted
6	Save the kept chunks and their hashes in the record manager	Future uploads are checked against it

Example output:

```
Before cleanup:  5,000 chunks
Duplicate groups: 1,900
Deleted copies:   2,300
After cleanup:   2,700 chunks  (46% smaller, no information lost)
```

```python
from collections import defaultdict

data = collection.get(include=["documents", "metadatas"])
groups = defaultdict(list)
for cid, text in zip(data["ids"], data["documents"]):
    groups[hashlib.sha256(text.strip().lower().encode()).hexdigest()].append(cid)

to_delete = [cid for ids in groups.values() for cid in ids[1:]]   # keep the first of each group
if to_delete:
    collection.delete(ids=to_delete)
print(f"Deleted {len(to_delete)} duplicate chunks")
```

Summary

Question	Answer
What happens if the same PDF is uploaded again with no protection?	Every chunk is stored twice; search results fill with copies; costs grow
How do I stop it before it happens?	Gatekeeper: file hash → content hash → `doc_id` → near-duplicate check
What if the gatekeeper is skipped?	Content-based chunk IDs + `upsert()` — a copy overwrites itself instead of adding
What if duplicates are already stored?	Hash all chunks, group identical ones, keep one, delete the rest
What does the uploader see?	"This document already exists — no changes made."

The next subsections explain each check in detail.

The Four Levels of "Duplicate"

Level	Example	How to Detect	Section
1. Exact file copy	Same PDF uploaded twice	File hash	4.1
2. Same content, different file	PDF and DOCX of the same policy	Content hash	4.2
3. Near-duplicate	One word or the footer changed	MinHash / SimHash	4.3
4. Same meaning, different words	Policy vs FAQ saying the same thing	Embedding similarity	4.4

Rule: Check in this order. The first checks are the cheapest.

4.1 Exact Duplicates — File Hashing

Simple Analogy

A fingerprint. Two identical files always have the same fingerprint. Change even one byte, and the fingerprint is completely different.

What It Is

A **hash** (SHA-256) turns a file into a fixed 64-character code. Same file → same code.

Step by Step

Step	What You Do	Example
1	A file is uploaded	`HR_Policy_2025 (1).pdf`
2	Calculate the SHA-256 hash of the file's bytes	`a3f9…7c21`
3	Look up the hash in the record manager (4.7)	Found — same as `HR_Policy_2025.pdf`
4	If found → skip. If not found → continue ingestion	Skipped

Example Output

File Uploaded	File Hash	Seen Before?	Action
HR_Policy_2025.pdf	`a3f9…7c21`	No	Ingest
HR_Policy_2025 (1).pdf	`a3f9…7c21`	Yes	**Skip — exact copy**
Travel_Policy.pdf	`81bd…02e4`	No	Ingest

Code

```
import hashlib

def file_hash(path):
    with open(path, "rb") as f:
        return hashlib.sha256(f.read()).hexdigest()
```

Limitation: If the same document is saved again (re-exported), hidden details like the save date change → the hash changes. Level 2 catches this.

4.2 Same Content, Different File — Content Hashing

Simple Analogy

The same letter printed on white paper and on cream paper. The paper is different, the words are the same. Content hashing looks at the **words**, not the paper.

What It Is

Hash the **cleaned text** of the document, not the file bytes.

Step by Step

Step	What You Do	Example
1	Extract the text from the file	"Sick Leave is 12 days per year. Page 3 of 25"
2	Clean it: lowercase, remove page numbers, headers, footers, extra spaces	"sick leave is 12 days per year."
3	Hash the cleaned text	`5b1e…90aa`
4	Look it up in the record manager	Found → skip

Example Output

File Uploaded	File Hash	Content Hash	Action
HR_Policy_2025.pdf	`a3f9…`	`5b1e…90aa`	Ingest
HR_Policy_2025.docx	`c77d…` (different)	`5b1e…90aa` (same)	**Skip — same content**
HR_Policy_2025_reexport.pdf	`e012…` (different)	`5b1e…90aa` (same)	**Skip — same content**

Catches: PDF vs Word copies, re-exported files, renamed and re-saved files.

4.3 Near-Duplicates — MinHash and SimHash

Simple Analogy

Two photocopies of the same page — one has a coffee stain. Not identical, but clearly 98% the same.

What It Is

A way to measure **how much** two documents overlap — quickly, even across thousands of documents.

Step by Step (MinHash)

Step	What You Do	Example
1	Break the text into small overlapping word groups ("shingles"), e.g., 3 words each	"sick leave is", "leave is 12", "is 12 days" …
2	Compare the word groups of two documents	Doc A and Doc B share 97 of 100 groups
3	Overlap score (Jaccard) = shared groups ÷ all groups	97 ÷ 100 = **0.97**
4	MinHash makes this fast: each document becomes a short signature (e.g., 128 numbers) that gives almost the same score	Compare 128 numbers instead of full text
5	Score above a threshold (commonly 0.8–0.9) → near-duplicate	0.97 → near-duplicate

Example

Document A	Document B	Overlap Score	Result
HR Policy 2025	HR Policy 2025 draft (one word changed: "two days" → "three days" work from home)	0.97	**Near-duplicate**
HR Policy 2025	Travel Policy	0.04	Different documents

SimHash (short version): Each document gets one 64-bit fingerprint. Similar documents have fingerprints that differ in only a few bits. Very compact — used for web-scale deduplication.

What to Do When a Near-Duplicate Is Found

Situation	Action
Near-copy of an existing document from the same source	Probably a new **version** → 4.6
Near-copy from a different source	Keep one copy; note the other as an alias
The small differences matter (e.g., Bengaluru vs Pune holiday list)	Keep both; add metadata `region: "Bengaluru"` / `"Pune"`

4.4 Semantic Duplicates — Embedding Similarity

Simple Analogy

Two newspaper reports of the same cricket match, written by different journalists. Different words, same story.

What It Is

Two chunks with **different words but the same meaning**. Found by comparing their embeddings (cosine similarity).

Step by Step

Step	What You Do	Example
1	Take two chunks	A: "Employees receive 12 days of paid sick leave annually." B: "You get twelve paid sick days every year."
2	Embed both	Two vectors
3	Calculate cosine similarity	0.96
4	Above a high threshold (e.g., 0.95) → semantic duplicate	Yes
5	**Tag** them with the same group ID — do not delete	`duplicate_group: "sick-leave-days"`

Why Tag Instead of Delete?

Reason	Example
The FAQ wording may match user questions better	Users type "sick days", not "sick leave annually"
Embeddings miss small number changes	"10 days" vs "12 days" can score 0.97 — deleting could remove the correct one

Rule: Delete only exact and confirmed near-duplicates. Handle semantic duplicates at search time with MMR (4.8).

4.5 Chunk-Level Deduplication

Simple Analogy

Two different books share the same chapter (for example, a legal disclaimer). You keep both books, but you do not need to shelve that chapter twice.

What It Is

Hash **each chunk**, not just each file. The same chunk inside different documents is stored only once.

Step by Step

Step	What You Do	Example
1	Clean and hash each chunk's text	Leave table chunk → `77a1…`
2	Check if that chunk hash already exists	Yes — it is in HR Policy
3	If yes: do not embed again; just add this document as another source	Sources: HR Policy, Onboarding Guide
4	If the chunk appears in very many documents (boilerplate), drop it	"This document is confidential…" in 40 files → dropped

Example Output

Chunk	Found In	Stored How Many Times?
Leave entitlement table	HR Policy, Onboarding Guide, Manager Handbook	**Once** (3 sources linked)
"About TechNova" paragraph	40 documents	**Dropped** — boilerplate
"Sick leave is 12 days per year."	HR Policy only	Once

Bonus: When a 100-page document changes on 2 pages, only the chunks on those 2 pages get new hashes. Only they need re-embedding.

4.6 Versioning — When the Same Document Is Updated

Simple Analogy

When a new edition of a textbook arrives, the new edition goes on the desk and the old one goes to the archive. Students never see both.

What It Is

A **duplicate** is the same content arriving twice. A **version** is changed content that **replaces** the old one.

The Key Idea: Every Document Needs a Stable ID

Field	Meaning	Example
`doc_id`	Which document is this? (never changes)	`hr-policy`
`doc_version`	Which edition	`2024` , `2025`
`is_current`	Is this the live version?	`true` / `false`

Where `doc_id` comes from: the source system's ID (SharePoint, Google Drive, Confluence — best), a naming rule, or the user choosing "this replaces…" when uploading.

Step by Step

Step	What You Do	Example
1	New file arrives with a known `doc_id`	`HR_Policy_2025.pdf` , doc_id = `hr-policy`
2	Find the current version in the record manager	Version 2024, 84 chunks
3	Split the new file into chunks and hash each chunk	86 chunks
4	Compare old chunk hashes with new chunk hashes	See table below
5	Keep unchanged, add new, remove (or archive) old	Only 7 chunks embedded
6	Mark version 2025 as current in the record manager	`is_current = true`

Example Output — Comparing Old and New Chunks

Chunk Status	Count	Action	Re-embed?
Unchanged (same hash in both)	79	Keep	No
New or changed (only in 2025)	7	Embed and add	Yes
Removed (only in 2024)	5	Delete, or mark `is_current = false`	No

One changed chunk:

Version	Text	Action
2024	"Sick leave is 10 days per year."	Removed / archived
2025	"Sick leave is 12 days per year."	Added

Result: The user asks "How many sick days do I get?" → only **12 days** is found.

Delete or Archive?

Option	When to Use
Hard delete old chunks	Most cases — simplest, no stale answers
Soft delete (`is_current = false` , filtered out when searching)	Users sometimes ask about history — "What was the policy in 2024?" (see 6.3)
Separate archive index	Regulated industries that need a full audit trail

---

4.7 Record Manager — Tracking What Is Indexed

Simple Analogy

A warehouse register. Every item that comes in is written down. Before accepting a delivery, the clerk checks the register.

What It Is

A small SQL database that remembers **every file, document and chunk** you have stored — with their hashes. The vector database stores vectors; the record manager keeps the list.

What It Stores

Table 1 — files

file_hash	file_name	doc_id	status
`a3f9…`	HR_Policy_2025.pdf	hr-policy	indexed
`a3f9…`	HR_Policy_2025 (1).pdf	hr-policy	skipped (exact copy)

Table 2 — documents

doc_id	version	content_hash	is_current
hr-policy	2024	`4c0e…`	false
hr-policy	2025	`5b1e…`	true

Table 3 — chunks

chunk_id	doc_id	chunk_hash	is_current
hr-policy::77a1	hr-policy	`77a1…`	true
hr-policy::09bc	hr-policy	`09bc…`	false

Step by Step — What Happens on Every Upload

Check (in order)	If Yes	Section
1. File hash seen before?	Skip — exact copy	4.1
2. Content hash seen before?	Skip — same content	4.2

Check (in order)	If Yes	Section
3. `doc_id` seen before?	Version update	4.6
4. Near-duplicate of another document?	Flag for review / treat as a version	4.3
5. None of the above	New document — full ingestion	3.8
For each chunk: chunk hash seen before?	Link — don't embed again	4.5

Framework support: LangChain's indexing API (with a `RecordManager`) and LlamaIndex's ingestion pipeline both do this bookkeeping. The rule is the same everywhere: **never write to the vector database without also writing to the register.**

4.8 Retrieval-Time Deduplication — MMR

Simple Analogy

At a buffet, the four closest dishes are all paneer. Each is good, but together it is a bad meal. You want the best dish, then the best **different** dish.

What It Is

MMR (Maximal Marginal Relevance) picks results that are **relevant** to the question **and different** from the results already picked.

Step by Step

Step	What You Do	Example
1	Retrieve more candidates than you need	Top 20 chunks
2	Pick the most relevant chunk first	"Sick leave is 12 days per year."
3	For each remaining chunk, score = relevance − similarity to what is already picked	FAQ "twelve sick days" scores low (repeats #1)
4	Pick the highest score	"Medical certificate needed beyond 2 days."
5	Repeat until you have K chunks	4 chunks

The formula: `MMR = λ × relevance − (1 − λ) × similarity to already-picked`

λ	Behaviour
1.0	Pure relevance (normal search)
0.5	Balanced — a common default
0.0	Pure diversity

Example Output — "What is the sick leave policy?"

Rank	Before MMR	After MMR (λ = 0.5)
1	"Sick leave is 12 days per year." (Policy)	"Sick leave is 12 days per year."
2	"You get twelve paid sick days a year." (FAQ) — *repeat*	"Medical certificate needed beyond 2 days."
3	"Sick leave: 12 days, no carry forward." (Table) — *repeat*	"Apply for sick leave through HRConnect within 48 hours."
4	"Medical certificate needed beyond 2 days."	"Unused sick leave lapses at year end."
Different facts	**2**	**4**

Most vector databases and frameworks have MMR built in (e.g., `search_type="mmr"` in LangChain).

SECTION 5 — METADATA, ACCESS CONTROL AND MULTI-TENANCY

Why This Section Is Required

When many kinds of people use one RAG system, **who is asking** matters as much as **what is asked**.

Situation	Risk Without Control
An engineer asks about salaries	Sees the VP's salary band
A Pune employee asks about holidays	Gets the Bengaluru holiday list
A SaaS product serves 200 client companies	Client A sees Client B's documents

The LLM will summarise **any** chunk it receives. So protection must happen **before** the chunks reach the LLM.

Simple analogy: An office ID card. Everyone enters the same lobby, but your card opens only the floors you are allowed on. Nobody relies on people "choosing not to look".

5.1 Metadata Schema Design

Simple Analogy

The label on a courier parcel: sender, address, weight, "fragile". Without the label, the parcel cannot be sorted, tracked or delivered to the right person.

What It Is

Deciding **which labels (metadata fields) every chunk carries** — before you index anything.

Step by Step

Step	What You Do	Example
1	List the questions and rules your system must support	"Show the source", "Hide salaries from engineers", "Latest version only"
2	For each need, choose a field	Source → `source_file`, `page`; Salaries → `access_level`; Latest → `is_current`
3	Fill the fields automatically during ingestion	From the file name, parser, source system permissions
4	Use the fields to filter while searching	`where access_level in [...] and is_current = true`

A Practical Schema

Category	Fields	Used For
Identity	`chunk_id`, `doc_id`, `doc_version`, `content_hash`	Dedup, versioning, updates
Location	`source_file`, `section`, `page`, `url`	Citations
Content	`element_type`, `language`, `department`, `topic`	Filters, routing
Time	`created_at`, `effective_from`, `effective_to`	Freshness (Section 6)

Category	Fields	Used For
Security	`access_level`, `allowed_groups`, `tenant_id`	Access control
Quality	`ocr_confidence`, `extraction_method`	Trust signals

Example Output — One Chunk With Its Labels

```
{
  "text": "Band L3 (Senior Engineer): Rs 12 lakh to Rs 18 lakh per year.",
  "metadata": {
    "doc_id": "compensation-policy",
    "doc_version": 2025,
    "is_current": true,
    "source_file": "Compensation_Policy_2025.pdf",
    "page": 1,
    "element_type": "table_row",
    "access_level": "hr_leadership"
  }
}
```

Tip: Decide the schema early. Adding a field later means re-processing every document.

5.2 Document-Level Access Control

What It Is

Every chunk carries **who is allowed to see it**. Every search is filtered by **who is asking**.

Step by Step

Step	When	What You Do	Example
1	Indexing	Copy the document's permissions onto every chunk	Salary table → `access_level: "hr_leadership"` ; Leave policy → `"all_employees"`
2	Query	Look up the user's groups from the login system	Rahul (engineer) → `["all_employees"]` ; Neha (CHRO) → `["all_employees", "hr_leadership"]`
3	Query	Add a filter **inside** the search	`where access_level in ["all_employees"]`
4	Query	Only permitted chunks are ranked and sent to the LLM	Salary chunks never reach the LLM for Rahul

Example Output — Same Question, Two Users

Question: "What is the salary range for L8?"

User	Groups	Chunks the Search Can See	Answer
Rahul (Engineer)	all_employees	Policies, FAQs — no salary table	"I could not find this in the documents you have access to."
Neha (CHRO)	all_employees, hr_leadership	Everything, including the salary table	"L8 (VP): Rs 70 lakh to Rs 1 crore per year."

Filter Before, Not After

Approach	How	Result
Pre-filter (recommended)	Search only among permitted chunks	Correct and safe
Post-filter	Search everything, then remove forbidden chunks	Top 4 may all be forbidden → empty answer; one bug = data leak

Rules:

- **Never** use the prompt for security ("Do not reveal salaries"). A prompt is not a lock.
- Sync permissions regularly. When someone leaves the HR group, their access must go immediately.
- Apply the same filter to **every** store — vector, SQL and graph.

5.3 Multi-Tenancy

Simple Analogy

An apartment building. The building is shared; each family's flat is locked.

What It Is

One RAG platform serving **many separate companies** (tenants). Each company's data must be completely hidden from the others.

Step by Step

Step	What You Do	Example
1	Give every tenant an ID	TechNova → `t_technova`; Acme → `t_acme`
2	Store each tenant's chunks in its own space	Namespace `t_technova`, namespace `t_acme`
3	When a user logs in, find their tenant	Priya → `t_technova`
4	Search **only** inside that tenant's space	Priya's question never touches Acme's data

Example Output — "What is the leave policy?"

User	Tenant	Searched	Answer From
Priya	TechNova	`t_technova` only	TechNova HR Policy
John	Acme	`t_acme` only	Acme Employee Handbook

Three Ways to Isolate Tenants

Pattern	How	Trade-off
Metadata filter (`tenant_id`)	One shared index; every query filtered by tenant	Cheapest; one missing filter = leak
Namespace / partition per tenant	Separate namespace inside one vector DB	Good balance — recommended for most SaaS
Separate database per tenant	Completely separate storage	Strongest isolation; highest cost

SECTION 6 — FRESHNESS AND INCREMENTAL INDEXING

Why This Section Is Required

Data changes every day — policies are revised, people join, prices change. A RAG system indexed once becomes more wrong every day, and it gives old answers with full confidence.

Simple analogy: A newspaper is printed every morning. But it does not reprint yesterday's unchanged articles — it only prints what is new.

6.1 Incremental Indexing

Simple Analogy

Updating a phone contact list. You don't delete all 500 contacts and type them again — you add the 2 new ones and edit the 1 that changed.

What It Is

On every sync, process **only what changed**: new files, changed files and deleted files.

Step by Step

Step	What You Do	Example
1	List all files in the source folder	3 files
2	Compare each file's hash with the record manager	See table below
3	**New file** → ingest it	Holiday_List_2026.pdf
4	**Changed file** → version update (only changed chunks re-embedded — 4.6)	Travel_Policy.pdf
5	**Unchanged file** → skip	HR_Policy.pdf
6	**File missing from the source** → remove its chunks	Old_Canteen_Menu.pdf

Example Output — Nightly Sync

File	Status	Action	Chunks Embedded
HR_Policy.pdf	Unchanged	Skip	0
Travel_Policy.pdf	Changed (per diem Rs 3,000 → Rs 3,500)	Update 1 chunk	1
Holiday_List_2026.pdf	New	Ingest	4
Old_Canteen_Menu.pdf	Deleted at source	Remove chunks	0
Total			**5** (instead of re-embedding everything)

Full Rebuild vs Incremental

	Full Rebuild	Incremental
Work done	Re-embed all 10,000 documents	Only the changed ones
Time	Hours	Minutes
Search during update	Broken or empty	Stays live

6.2 Change Data Capture (CDC)

Simple Analogy

Instead of checking your mailbox every hour, you get a notification the moment a letter arrives.

What It Is

Instead of checking all files on a schedule, the source system **tells you** when something changes (a webhook or change event). You update only that item, within seconds.

Step by Step

Step	What Happens	Example
1	Register a webhook with the source system	SharePoint: "notify me when a file in /HR changes"
2	A user edits a document	HR updates Travel_Policy.docx at 3:05 PM
3	The source sends a change event	`{file: "Travel_Policy.docx", event: "updated"}`
4	Your pipeline runs the version update for that one file (4.6)	1 chunk re-embedded
5	The new answer is live	At 3:06 PM, users see Rs 3,500 per diem

Scheduled Sync vs CDC

	Scheduled Sync (6.1)	CDC
How it knows about changes	Checks everything on a timer	Is told instantly
Delay	Up to the next run (e.g., next night)	Seconds
Complexity	Simple	Needs webhooks / event queue

Which to Use

How Often the Data Changes	Best Approach
Monthly (policies, manuals)	Nightly incremental sync
Hourly (wikis, tickets)	Frequent sync or CDC
Every second (headcount, prices, stock)	**Do not embed** — query the live SQL database at question time

Rule: Numbers that change often should not be embedded. Embed only a *description* of where the number lives, and query the live source.

6.3 Time-Aware Retrieval

Simple Analogy

A notice board where every notice has a "valid from" and "valid until" date. Expired notices are taken down, and the office keeps a file of old notices in case someone asks.

What It Is

Using **dates and versions** in metadata so that:

- normal questions get the **current** answer, and
- history questions ("in 2024?") get the **old** answer.

Step by Step

Step	What You Do	Example
1	Store time fields on every chunk	`effective_from: 2025-04-01`, `effective_to: null`, `doc_version: 2025`
2	Read the question — does it mention a date or year?	"What **was** the sick leave in **2024**?" → year 2024
3	No date → filter to current chunks	`is_current = true`
4	Date found → filter to chunks valid at that time	`doc_version = 2024`
5	Search and answer	See table below

Example Output

Question	Filter Used	Answer
"How many sick days do I get?"	`is_current = true`	12 days (2025 policy)
"What was the sick leave in 2024?"	`doc_version = 2024`	10 days (2024 policy)
"What is the holiday list?"	`effective_from ≤ today ≤ effective_to`	2026 list (the expired 2025 list is excluded)

Three Techniques

Technique	How It Works	Example
Validity filter	Only chunks valid today	Expired holiday list never shown
Recency boost	Newer chunks get a small score bonus	Latest support ticket ranks above a year-old one
Explicit time query	Year in the question → search that version	"Policy in 2024?" → archived 2024 chunks

Caution: Do not boost recency for everything. A 2021 policy that is still valid should not lose to a 2025 blog post that only mentions the topic.

SECTION 7 — WHEN TO USE WHAT

By Problem

Problem You Are Facing	Best Technique
Questions about who reports to / depends on / connects to whom	Graph RAG — Text-to-Cypher or neighbourhood expansion
Multi-hop questions fail	Graph RAG
"What are the main themes across all documents?"	GraphRAG global search or RAPTOR
Need both text detail and relationships	Hybrid Graph + Vector RAG
Same entity appears under different names	Entity resolution
Tables come out as word soup	Layout-aware parsing + table strategy
Questions need calculations over tables	Load tables into SQL + Text-to-SQL
Charts and diagrams hold key facts	Vision LLM captioning
Slide decks, infographics, visual reports	Page-image retrieval (ColPali style)
Scanned documents	OCR with confidence metadata
Spreadsheets	SQL + sheet descriptions in the vector store
Recordings and meetings	Transcription + diarisation + timestamps
Same file uploaded repeatedly	File hash + content hash
Slightly edited copies	MinHash / SimHash

Problem You Are Facing	Best Technique
Same boilerplate everywhere	Chunk-level dedup
Old and new policy versions contradict	doc_id + versioning + upsert/cleanup
No idea what is already indexed	Record manager
Top-K full of near-identical chunks	MMR
Users see data they should not	Pre-filtered metadata access control
Multiple client companies on one platform	Namespaces / multi-tenancy
Answers based on outdated data	Incremental indexing, CDC, live SQL for numbers

By Stage of the Pipeline

```
Before indexing:
  Clean the input  → Deduplication (file, content, near, chunk)
  Parse properly   → Layout-aware parsing, OCR
  Handle modality  → Table strategies, image captioning,
                     page-image retrieval, transcription
  Choose the store → Vector | SQL | Graph (often all three)

During indexing:
  Structure        → Entity extraction, entity resolution,
                     community summaries
  Label            → Metadata schema, access control, versions
  Track            → Record manager

During retrieval:
  Secure           → Pre-filtered access control, tenant isolation
  Stay current     → Validity filters, time-aware retrieval
  Connect          → Graph traversal, hybrid graph + vector
  Diversify        → MMR
```

Starting Point Recommendation

Do not build all of this at once. Most of it is only worth its cost once a specific failure shows up.

```
Step 1: Fix the data before fixing the retrieval:
        1. Deduplication + record manager    (cheap, prevents mess)
        2. Versioning with doc_id            (prevents contradictions)
        3. Metadata + access control         (non-negotiable for real users)
        4. Layout-aware parsing + tables     (biggest hidden accuracy win)

Step 2: Add modalities your data actually contains:
        5. Image captioning for charts/diagrams
        6. SQL for spreadsheets and numeric tables

Step 3: Add structure only when questions demand it:
        7. Graph RAG — when you have real multi-hop questions
        8. GraphRAG global search — when "big picture" questions matter

Step 4: Keep it current:
        9. Incremental indexing / CDC
```

> Section 12 taught you to **retrieve** better. Section 13 teaches you to **store**, **ingest** and **connect** data better. In production, that is usually where the biggest wins are hiding.

Suresh D R — AI Product Developer & Technology Mentor

Machine Learning and AI Complete Guide — Section 13: Production RAG — Graphs, Multimodal Data & Data Hygiene